Mind the Windows: Row-Level Non-IID Partitioning Inflates Heterogeneity Effects in Federated Time-Series Benchmarks An O-RAN Slice-SLA Case Study and Correction

Hsiu-Chi Tsai, *Student Member, IEEE*

Abstract*—Federated learning (FL) on O-RAN edge gNBs is a natural candidate for slice-level SLA-violation forecasting when raw per-user telemetry is undesirable or infeasible to pool centrally (FL itself does not provide formal privacy guarantees; see Section VIII L16). Yet the deployment design space — sequence backbone, federation algorithm, client-heterogeneity partition, commodity edge hardware — has been studied one axis at a time, leaving the joint behaviour under realistic O-RAN traffic characterised only anecdotally. This paper presents the first cross-architecture empirical FL benchmark on the public Colosseum/ColO-RAN testbed: 3 backbones (LSTM, Mamba, Spiking-SSM) $\times$ 5 algorithms (FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn) $\times$ 6 partitions (natural-by-BS + Dirichlet $\alpha \in \{0.05..5.00\}$) $\times$ 10 seeds $=$ 900 cells, with NVML idle-baseline-subtracted training-energy measurement on RTX 4080 and controlled run-level vs row-level partitioning experiments on $4\times$ V100-SXM2-32GB that diagnose the apparent inverted-α as a sequence-integrity artifact. Four findings: (1) an apparent "inverted heterogeneity" — the natural base-station partition seemingly beating every Dirichlet α — is a sequence-integrity artifact: the de-facto FL non-IID partitioning (row-level Dirichlet/random assignment) scatters each run's consecutive timesteps across clients and corrupts the per-client sliding windows, while natural-by-BS happens to keep runs intact; a run-level Dirichlet control at matched α recovers full performance and erases the inversion (the per-client contiguous-window fraction rank-correlates perfectly with AUC), so the inversion is a benchmarking artifact, not a RAN structural property — a previously-unflagged pitfall for federated time-series benchmarks, for which we give the diagnosis, a run-level fix, and a corrected benchmark; (2) server-side algorithmic gains are small — FedAdam improves over FedAvg by at most $+0.016$ AUC in the Dirichlet sweep, and a 5-seed FedSWA probe at LSTM $\times$ natural-by-BS exceeds FedAdam by only $+0.0004$ AUC, suggesting no large LookAhead-style gain in this representative cell; (3) selective-state-space backbones interact catastrophically with control-variate algorithms at moderate skew (Mamba $\times$ SCAFFOLD $\times$ $\alpha \in \{0.10, 0.50\}$); (4) architecture choice traverses $10\times$ in commodity-GPU energy and an order of magnitude larger AUC range than algorithm choice. A five-architecture extension (adding xLSTM and Mamba-3) exhibits the same apparent advantage, as expected of an artifact that is architecture-agnostic by construction (corrupted sliding windows degrade any sequence backbone). We provide a reference benchmark with paired-bootstrap CI_{95} over 270 paired distributions, NVML-instrumented per-cell energy, and a preregistered statistical protocol, released under AGPL-3.0 with Croissant metadata and archived at Zenodo (concept DOI 10.5281/zenodo.20263144).

H.-C. Tsai is with the Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu, Taiwan. E-mail: hctsai1006@cs.nctu.edu.tw. ORCID: 0000-0001-7421-8027.

***Index Terms*—Federated learning, O-RAN, slice management, SLA prediction, state-space models, spiking neural networks, NVML energy measurement, paired-bootstrap inference.**

I. Introduction

OPEN Radio Access Networks (O-RAN) decomposes the cellular base station into commodity hardware components controlled by ML-driven xApps and rApps running on disaggregated RAN Intelligent Controllers (RICs) [1]. Slice-level Service Level Agreement (SLA) violation forecasting is one near-RT RIC xApp pattern of practical interest: each gNB simultaneously serves heterogeneous traffic slices (the ColO-RAN release exercises eMBB constant-bitrate, MTC Poisson, and URLLC Poisson — see Section III), and proactive forecasting of next-second SLA risk informs near-RT RIC resource-allocation decisions within its 10 ms – 1 s control loop. Because edge gNBs accumulate per-user telemetry whose central pooling is undesirable or infeasible (operator policy, regulatory privacy, backhaul bandwidth), federated learning (FL) is a natural training paradigm: each gNB trains a local sequence model on its own traces and shares only weight updates with the FL aggregator. We place the aggregator in the non-RT RIC / SMO orchestration layer ($\geq$ 1 s timescale fits our 100-round federated training cadence per Section IV); a near-RT-RIC-internal xApp aggregator would alternatively be possible if cross-cell xApp messaging supports the federation protocol — we do not engage with that deployment alternative beyond noting it.

Standard FL practice — codified by a decade of toy-image benchmarks — treats client heterogeneity as a *wound* to be healed: lower Dirichlet α (more skew) drives lower accuracy, motivating sharpness-aware methods, control variates, and proximal regularizers. **In O-RAN slice SLA prediction, this premise *appears* to invert** — for an instructive reason we unpack below. When clients are the *natural* base stations of a Colosseum/ColO-RAN testbed (one client per gNB), the federated average across 7 base-station-natural clients consistently outperforms every uniform-Dirichlet partition of the same training data on the same model + algorithm. Empirically (Section VI, all values are out-of-sample test AUC averaged over 10 seeds), the natural-by-BS partition achieves AUC 0.913–0.918 on LSTM (5-algo range; $\Delta \approx 0.005$), 0.8998–0.9186 on Mamba (with SCAFFOLD as the lower outlier

and the other four algorithms in 0.911–0.919), and 0.840–0.856 on Spiking-SSM, **strictly above the AUC obtained at any parametric Dirichlet** α for every (arch, algo) pair, and monotonically increasing as $\alpha \to 0$ within the Dirichlet sweep. The $\alpha = 5.0 \to \alpha = 0.05$ AUC gap is ≈ 0.10–0.12 on the dense LSTM/Mamba backbones; on Spiking-SSM the gap is smaller (≈ 0.022) but still positive, and across all five algorithms heterogeneity is associated with higher AUC, not lower. We initially read this as evidence of cell-conditional structure preserved by bs grouping. Controlled run-level partitioning experiments (Section VII-A) overturn that reading: the collapse under Dirichlet or `random_split` is not about losing bs grouping per se but about **row-level partitioning scattering each run's consecutive timesteps across clients**, which corrupts the per-client sliding windows the sequence models depend on. A run-level Dirichlet partition (intact runs, Dirichlet-skewed) at matched α recovers natural-by-BS AUC and erases the inversion; breaking bs-coherence while keeping runs intact (`run_random`) costs $\sim$0 AUC (paired CI_{95} $[-0.0003, +0.0001]$). The apparent inversion is therefore a **sequence-integrity artifact of how the non-IID partitions are constructed**, not a structural property of RAN telemetry. The deployment recommendation is unchanged and trivial — partition by natural gNB, which preserves each cell's time series — but the *scientific* reading that RAN telemetry has special cell-conditional structure defeating standard FL heterogeneity does not survive the controls.

Despite the explosion of FL benchmarks since 2018, the deployment design space for FL-based slice SLA prediction remains under-characterized. The space spans four axes:

1) **Sequence model architecture** — recurrent (LSTM), structured state-space (Mamba), or bio-inspired sparse spiking variants (Spiking-SSM).
2) **Federation algorithm** — FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn (and the more recent sharpness-aware family discussed in Section II).
3) **Client heterogeneity** — including the natural base-station partition (one gNB $\Rightarrow$ one client) and parametric Dirichlet stress $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ over slice IDs.
4) **Commodity hardware** — consumer-tier edge GPUs (RTX-class) at the gNB rather than data-center accelerators.

Existing benchmarks address each axis in isolation. NeurIPS-class FL benchmarks evaluate algorithm $\times$ heterogeneity on toy image data [2], [3], [4], [5]. Spiking-NN benchmarks compare architectures in centralized regimes on neuromorphic-friendly datasets [6]. FL $\times$ wireless papers focus on a single architecture [7], [8], [9]. The implicit assumption underlying single-axis recommendations: **best architecture $\times$ best algorithm $\times$ representative α $\times$ commodity hardware = best deployment**.

This paper tests the assumption empirically. We construct the first comprehensive 4-axis FL benchmark on the publicly-available Colosseum/ColO-RAN testbed dataset [1], [10], sweeping {LSTM, Mamba, Spiking-SSM} $\times$ {FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn} $\times$ {natural-by-BS, Dirichlet $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$} $\times$ 10 seeds = 900 cells (90 groups, all 10 seeds populated), with NVML-instrumented per-round GPU energy measurement on RTX 4080 (sm_89) commodity hardware. Statistical inference uses paired-bootstrap CI_{95} on per-seed AUC deltas; Wilcoxon p-values are reported uncorrected per finding (with family-wise Bonferroni discussed in Section IV-E as a supplementary check).

Contributions:

1) **A sequence-integrity pitfall in federated time-series benchmarking (primary contribution)**: we first observe an apparent "inverted heterogeneity" — the natural base-station partition outperforms every parametric Dirichlet α across all architectures and algorithms, contradicting the standard "skew hurts" assumption. Through controlled, same-environment, paired run-level partitioning experiments (`run_random`; run-level Dirichlet) we show this is a **sequence-integrity artifact**: the standard row-level non-IID partitioning scatters each run's consecutive timesteps across clients, so per-client sliding-window construction yields temporally-broken windows; natural-by-BS merely keeps runs intact. With intact sequences AUC is flat across heterogeneity and coherence (run-level Dirichlet $\approx$ natural at every α; breaking bs-coherence with intact runs costs $\sim$0 AUC, paired CI_{95} $[-0.0003, +0.0001]$; shown for LSTM, confirmed on Mamba and Spiking-SSM in Section VII-A), while the row-level deficit and its α-dependence track window-fragmentation severity. Because row-level Dirichlet partitioning is the de-facto FL-benchmark default (NIID-Bench) and is routinely transplanted to federated time-series tasks, this pitfall plausibly affects other federated time-series benchmarks; we provide the diagnosis, a sequence-preserving (run-level) partitioning fix and a checklist, and a corrected O-RAN slice-SLA benchmark.
2) **Algorithmic leverage is small in this benchmark.** Among five preregistered FL algorithms, FedAdam is best but improves over FedAvg by at most $+0.016$ AUC (Dirichlet-averaged) in our sweep; per-architecture paired-bootstrap CI_{95} details in Section VI-C. A 5-seed FedSWA probe on LSTM $\times$ natural-by-BS (Section VII-E) changes the ranking by only $+0.0004$ AUC $[+0.0002, +0.0006]$, and FedBN [11] is structurally equivalent to FedAvg for our no-BatchNorm backbones (proof + 30-cell empirical confirmation in Section VIII L2). A 60-cell extension with FedSCAM [12] and FedGMT [13] (Section VII-F) finds FedSCAM extracts a $+0.0092$ AUC paired gain at the moderate-heterogeneity $\alpha_{\text{dir}} = 0.10$ cell (5/5 same-direction, CI_{95} excludes zero) but ties FedAvg at IID and extreme α_{dir}, while FedGMT is strictly dominated by FedAdam across all six partitions tested ($\Delta \in [-0.027, -0.005]$). These results suggest that, in this small-N high-participation regime, architecture and partition choices dominate server-side optimiser choice, with one moderate-heterogeneity niche for per-client

SAM radius modulation. FedMoSWA [14] empirical evaluation on Colosseum/ColO-RAN remains deferred to follow-up work; the mechanism-based discussion is in Section VII-B.

3) **Architecture $\times$ algorithm catastrophic interaction (Mamba $\times$ SCAFFOLD $\times$ $\alpha \in \{0.10, 0.50\}$)**: SCAFFOLD on Mamba at $\alpha = 0.10$ yields AUC 0.7609 ± 0.0830 — std amplified $\sim 3\times$ over (Mamba, FedAvg) at the same α (0.0251) and $\sim 10\times$ over (Mamba, FedAvg) at $\alpha = 5.0$ (0.0073). The deployment warning is concrete: control-variate methods interact pathologically with selective-scan SSMs under high heterogeneity, even as both components individually behave well on neighboring cells.
4) **Architecture leverage dominates algorithm leverage** (paired-bootstrap CI_{95}) **on RTX 4080**: on the energy-AUC Pareto front (Section VI), architecture choice spans $10\times$ in NVML-measured model-attributable energy on RTX 4080 (LSTM $\sim$ 3.5 kJ $\rightarrow$ Mamba $\sim$ 10.5 kJ $\rightarrow$ Spiking $\sim$ 41 kJ per cell). Holding (algorithm, IID partition) fixed, the paired Spiking $-$ LSTM AUC delta is concentrated at -0.061 to -0.074 across all 5 algorithms (all 5 cells significant at $p < 0.005$), substantively dwarfing the FedAdam $-$ FedAvg algorithmic gap of $+0.006$ to $+0.016$ on the same architectures (contribution 2). The $10\times$ span is hardware-specific and **implementation-specific**: on a sparsity-aware accelerator (Loihi-2, IBM NorthPole) the LSTM-vs-Spiking ratio would compress; and within our pure-PyTorch implementation, Mamba uses a sequential-scan kernel rather than the optimised `mamba-ssm` Triton kernel, which would tighten the LSTM-vs-Mamba gap on this implementation but does not affect the LSTM-vs-Spiking band-separation conclusion (Section VI-E + Section VIII L1, L6). Operator decisions on this RTX 4080 implementation should centre architecture-energy trade-off, not algorithm breadth.
5) **Open reproducible reference benchmark**: spec-driven YAML pipeline with pre-flight algorithm validation and `--skip-completed` resumability; paired-bootstrap statistical pipeline; NVML idle-baseline-subtracted training energy; Croissant metadata; Zenodo-archived (concept DOI 10.5281/zenodo.20263144); AGPL-3.0 license (preregistered). The full 4-axis table maps deployment configurations $\rightarrow$ (AUC, F1, energy, paired CI_{95}) over 90 (arch $\times$ algorithm $\times$ partition) groups, enabling lookup-style operator decisions on Colosseum/ColO-RAN.

Positioning against concurrent work. Our contribution is empirical multi-architecture characterisation on RAN telemetry, not the proposal of a new federated algorithm or backbone. **FedRMamba** [15] (WWW 2026, concurrent submission) is the closest contemporary method-proposal in the federated-Mamba time-series space; it differs from our work in contribution type (method-design vs cross-architecture benchmark) and domain (general multivariate forecasting vs RAN slice-SLA prediction) — we discuss the differentiation in detail in Section II-G. We do not claim first-of-kind status for the individual federated Mamba / xLSTM / Mamba-3 instances; we claim the first comprehensive cross-architecture FL benchmark on the publicly-available Colosseum/ColO-RAN testbed with paired-bootstrap inference and per-round NVML energy. We additionally extend the 3-architecture core panel with xLSTM [16] and Mamba-3 [17] as a robustness check against the most recent recurrent / state-space backbones; the 5-architecture panel is reported in Section VI-I.

The remainder of the paper is organized as follows. Section II reviews related work across the four axes. Section III describes the Colosseum/ColO-RAN dataset and our preprocessing pipeline. Section IV details the architecture, algorithm, and partition methodology. Section V enumerates the reproducibility infrastructure and our Croissant-aligned data release. Section VI reports results across the 90 (arch $\times$ algorithm $\times$ partition) groups with paired-bootstrap CI_{95}. Section VII discusses the mechanism finding and articulates the applicability boundary on commodity edge hardware. Section VIII enumerates limitations. Section IX concludes.

II. Related Work

A. Federated learning benchmarks

The canonical FL benchmark is **LEAF** [2], which standardized federated MNIST/Shakespeare evaluation pipelines for early FedAvg variants. **Flower** [18] later provided a framework rather than a benchmark, supporting heterogeneous device experiments. **FedScale** [3] scaled to thousands of cross-device clients; **FLAIR** [4] introduced a long-tail multi-label image dataset. **pfl-research** [5] integrates differential-privacy mechanisms into a simulation framework. Most recently, **fev-bench** [19] introduced rigorous bootstrap-CI_{95} evaluation for time-series forecasting, though without the federated dimension. None of these benchmarks targets O-RAN slice SLA prediction specifically; their toy-data settings cannot capture the covariate-skew structure of real RAN telemetry.

B. FL on cellular and wireless networks

A first generation of FL $\times$ cellular work targeted SLA-constrained slicing under the Beyond-5G banner. **Statistical FL for B5G SLA-Constrained RAN Slicing** [20] introduced a long-term CDF SLA constraint. **CDF-Aware FL** [21] formalized the constraint as a per-client penalty. **Fed-LSTM** [7] deployed an LSTM forecaster for slice-level traffic. **TRACTOR** [22] integrated reinforcement-learning xApps for resource-block assignment. **REAL** [23] integrated the OSC near-RT RIC with srsRAN for RL-based slice-resource-allocation xApps under urban-mobility channel modelling. Recently **Hayek et al. 2025** [24] measured FL convergence on a 5G/WiFi/Ethernet COTS testbed with Raspberry-Pi clients, exposing uplink straggler effects. **arXiv:2508.08479** [8] benchmarked FedAvg/FedProx/FedBN on five throughput-prediction datasets with LSTM, CNN, CNN+LSTM, and Transformer architectures, finding FedBN superior under feature skew. **Mangi et al. 2026** [9] ("When Clients Drift") proposed regime-aware aggregation under leave-one-regime-out evaluation on synthetic 6G RAN telemetry.

Each of these papers covers one or two of our four axes. None benchmarks LSTM, Mamba, and Spiking-SSM jointly under parametric Dirichlet heterogeneity, and none reports per-round NVML energy. Our work is differentiated by the combination of architecture breadth, algorithm breadth, parametric heterogeneity, and energy instrumentation on the **publicly-available** Colosseum/ColO-RAN dataset, in contrast to closed simulators or undisclosed datasets used by Mangi et al.

C. Spiking-SSM and FL × spiking neural networks

Hybrid spiking-state-space architectures emerged from late 2024. **SpikMamba** [25] combined LIF spiking neurons with Mamba selective scan for event-camera action recognition. **Mamba-Spike** [26] introduced a spiking front-end for general temporal data. **Spiking Point Mamba** [27] extended to 3D point clouds. **SpikingSSMs** [28] formalized sparse-parallel spiking state-space layers. **SpikingMamba** [29] distilled large language models into spiking variants for energy efficiency. **SpikingBrain** [30] scaled spiking architectures to 76B parameters on data-center GPU clusters with explicit FLOP-utilization measurement.

Federated-learning variants of spiking neural networks emerged earlier than spiking-SSM: **arXiv:2106.06579** [31] founded FL × SNN. **FedLEC** [32] extended to label-skew non-IID. **SNN in vertical FL** [33] examined VFL energy trade-offs. **Robustness of SNN in FL with compression** [34] addressed Byzantine attacks. **Privacy in FL with SNN** [35] characterized gradient leakage. Most recently, **arXiv:2602.12009** [36] examined firing-rate sensitivity to differential privacy in federated SNN — preempting the DP-SNN-FL angle. We accordingly cite this work in Section VII as a deferral and do not duplicate the DP study; our contribution is orthogonal in its multi-architecture × parametric-Dirichlet × NVML scope.

To our knowledge, no published work combines spiking-SSM with FL on cellular/RAN telemetry. Our `spiking_expand2` variant adapts the wide-inner-state design pattern from Mamba [37] to spiking-SSM and demonstrates competitive AUC/energy trade-off on commodity edge GPU under FL.

D. GPU energy measurement methodology

The argument that FLOP counts mis-estimate actual hardware energy predates our work. **Shen et al. 2023** [38] ("Is Conventional SNN Really Efficient?") critiqued synaptic-operation accounting via a Bit Budget framework, finding that quantized ANNs dominate SNNs at equivalent bit budgets. **α-FLOPs** [39] proposed a parallelism-aware alternative. **NeuroBench** [40] standardized neuromorphic benchmarking on real hardware. The **ML.ENERGY Benchmark** [41] measured inference energy on generative AI; **Where Do the Joules Go?** [42] diagnosed the breakdown across model families. **STEP** [6] specifically benchmarked spiking transformers including memory-access cost. **Beyond Backpropagation** [43] applied NVML and CodeCarbon to alternative training objectives.

We follow the **Energy API** approach (`nvmlDeviceGetTotalEnergyConsumption`, available on Volta+ with NVML $\geq$ 11.0) preferred over polling-power Riemann-sum integration per the ML.ENERGY 2025 paper [41] and accompanying leaderboard. Our novel angle is the **federated-training regime**: prior NVML benchmarks measure inference [40], [41] or centralized training [30], [43]; we measure per-round energy across an FL sweep, including idle-baseline subtraction.

E. Heterogeneity in federated learning

NIID-Bench [44] established Dirichlet partitioning as the standard non-IID benchmark. **FedBN** [11] keeps client-local BatchNorm statistics for feature-skew settings; **arXiv:2508.08479** [8] demonstrated FedBN's superiority on cellular feature-skew. **pFedFDA** [45] specifically targets covariate shift. **Borazjani et al. 2025** [46] argued for embedding-based heterogeneity definitions beyond label-skew.

We use Dirichlet partitioning over slice IDs as the canonical covariate-skew model for O-RAN: each gNB serves a different mix of slice traffic, with mixing entropy controlled by α_{dir}. We adopt $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ to span extreme stress (each gNB dominated by a single slice) through near-IID. We additionally evaluate the natural base-station partition (one gNB $\Rightarrow$ one client, 7 ColO-RAN gNBs in the public release) as a non-parametric heterogeneity reference. We discuss the applicability of embedding-based alternatives in Section VII.

F. Sharpness-aware federated learning (2024–2026)

A separate strand of recent FL work targets *heterogeneity-induced sharp minima* via per-step learning-rate cycling and server-side weight extrapolation. **FedSAM** [47] adapted Foret et al.'s SAM perturbation to the federated client step. **FedSWA** [48] introduced an intra-round cyclical local LR schedule (eq. 3) combined with a LookAhead-style server EMA $\theta_t = \theta_{t-1} + \alpha(v_t - \theta_{t-1})$ with $\alpha = 1.5$ (eq. 17). **FedSCAM** [12] composed SAM with clustering for further sharpness regularization. The reported gains (FedSWA: $+5.6\%$ over FedAvg on CIFAR-100 with 10% client participation across 1000 rounds) assume large-N low-participation FL with coherent client drift toward sharp minima.

We do not include this family in the main 5-algorithm pre-registered baseline; an empirical 5-seed paired test of FedSWA at LSTM × natural-by-BS is reported in Section VII-E, and a 60-cell extension to FedSCAM + FedGMT [13] across IID + 5 Dirichlet alphas is reported in Section VII-F (FedMoSWA remains deferred). Two structural mismatches make the original absence-from-baseline decision defensible on mechanism alone. First, our regime — 7 base-station-natural clients with 71% per-round participation, 100 rounds, RAN slice SLA prediction — is *aggregation-noise-dominated* rather than coherent-drift-dominated. Second, FedAdam's adaptive variance damping $\mathrm{Adam}(m, v)$ over $(v_t - \theta_{t-1})$ systematically dominates FedSWA's variance-blind LookAhead overshoot *in expectation* in this regime: FedAdam knows when to step small (high v) and when to step normally (low v), while FedSWA's

LookAhead extrapolation rate α_{LA} (the FedSWA paper's α-symbol; we write it α_{LA} throughout to disambiguate from the Dirichlet α_{dir} of Section IV-C and the FedDyn α_{dyn} of Section IV; FedSWA's preregistered value is $\alpha_{\mathrm{LA}} = 1.5$) amplifies *both* signal and noise uniformly. Empirically (Section VI-C), FedAdam already saturates the headroom available to server-side aggregation modifications at $+0.006$–$+0.016$ Dirichlet-averaged AUC over FedAvg across the 3 architectures (paired-bootstrap CI_{95} in Section I contribution 2); LookAhead overshoot is unlikely to *systematically* exceed this ceiling without the variance-estimation machinery FedAdam has and FedSWA lacks. We additionally note that FedSWA targets *heterogeneity-as-sharpness-wound*, which our inverted-α finding (Section I, Section VI) indicates does not characterize this dataset.

G. Concurrent Mamba-in-FL and recent recurrent / state-space work (2025–2026)

Our benchmark is contemporary to a growing body of federated state-space-model methodology, and we position our contribution explicitly against it. **FedRMamba** [15] (WWW 2026, concurrent submission) proposes a personalised federated forecasting framework in which each client combines a global frequency-aware Mamba module with a local patch-wise Mamba module for multivariate time-series. Their contribution is *method-proposal* for general multivariate forecasting; ours is *multi-architecture empirical characterisation* on RAN telemetry with paired-bootstrap statistical inference — the two contribution types are complementary rather than competing, and we do not duplicate their architectural innovation.

A separate strand concurrently advances individual recent architectures in wireless and cellular settings. Ali et al. [49] (July 2025) apply the xLSTM sLSTM block to *centralised* cellular traffic forecasting; we extend the same block to the federated O-RAN slice-SLA prediction setting in Section VI-I. **WiMamba** [50] (March 2026) positions Mamba as a wireless foundation model, supporting our state-space backbone inclusion. **SliceFed** [51] (March 2026) targets federated O-RAN slicing via multi-agent DRL for spectrum-*control* rather than supervised SLA *prediction* — adjacent scope, different task.

We therefore do *not* claim first-of-kind status for the individual federated Mamba / xLSTM / Mamba-3 instances. Our novelty is the specific combination: comprehensive cross-architecture FL benchmark on the publicly-available Colosseum/ColO-RAN testbed with paired-bootstrap inference, per-round NVML idle-subtracted energy, and two contrarian empirical findings (inverted-α_{dir} heterogeneity and the Mamba $\times$ SCAFFOLD $\times \alpha_{\mathrm{dir}} \in \{0.10, 0.50\}$ catastrophic interaction; Sections VI-B and VI-D). A future paired-baseline 5-architecture extension (running xLSTM / Mamba-3 against FedAvg / FedAdam) is open work and is flagged in Section VI-I.

III. Dataset

A. Colosseum / ColO-RAN testbed

ColO-RAN [1] is the publicly-released RAN telemetry corpus collected from the Colosseum digital-twin testbed [10], comprising the per-second downlink/uplink physical-layer measurements of 7 base stations (Colosseum Rome scenario, BS nodes $\{1, 8, 15, 22, 29, 36, 43\}$; we re-index these to `bs_id` $\in \{1, \ldots, 7\}$ for ease of use) operating concurrently under 28 distinct traffic configurations (`tr` $\in \{0, \ldots, 27\}$, each a different RBG-per-slice allocation pattern; the exact (slice 0 / slice 1 / slice 2) RBG triples per `tr` are documented in the ColO-RAN release). Each base station serves 3 logical slices, and each slice can be served by one of 3 scheduling policies; both index mappings are listed in Table I. The unified telemetry table contains ≈ 18 M rows of (timestamp, `bs_id`, `slice_id`, `sched`, `tr`) keyed observations with 17 continuous features (uplink/downlink throughput, MCS/CQI, BLER, RB utilisation, buffer occupancy; full list in Section IV). We use the public release [52] without modification; no synthetic augmentation, no re-simulation. The OOD train/val/test split by traffic-config index `tr` (`tr` $\in \{0..21\}$ train, `tr` $\in \{22..24\}$ val, `tr` $\in \{25..27\}$ test) is a leave-future-traffic-configs-out design, simulating deployment to unseen RBG allocation patterns.

TABLE I
ColO-RAN slice and scheduler index mapping.

Index	Slice ID	Traffic class	Workload
0	eMBB	broadband	4 Mbps CBR/UE
1	MTC	machine-type	Poisson 30 pkt/s, 125 B/UE
2	URLLC	ultra-reliable	Poisson 10 pkt/s, 125 B/UE
Index	**`sched` ID**	**Scheduler**	
0	RR	Round-Robin	
1	WF	Waterfilling	
2	PF	Proportionally Fair	

B. Task definition: per-slice SLA-violation forecasting

The forecasting target is the binary indicator $y_{t+1} = \mathbf{1}[ul\_bler_{t+1} > 0.10]$, predicting whether the next-second uplink BLER on a given (`bs_id`, `slice_id`) tuple will breach the 10% SLA threshold. This threshold is the canonical SLA-violation gate used by the ColO-RAN authors [1]; the binary framing keeps the metric (AUC, F1) decision-relevant for the near-RT RIC's resource-allocation xApps. The empirical positive rate on the train partition is 30.9% (per-slice 34.9% / 26.7% / 31.0%, measured on `tr` $\in \{0..21\}$; see Section VII for how this rate enters the per-client loss reweighting). Inputs to the model are the most recent five seconds of (`slice_id`, `sched`, `tr`, 17 continuous features) on the same (`bs_id`, `slice_id`) tuple — `seq_len=5` is preregistered from v3/v4.

C. Out-of-distribution split by traffic config

To prevent label leakage between train / validation / test, we partition the 28 traffic configurations into three disjoint groups: train `tr` $\in \{0..21\}$ (22 configs), validation `tr` $\in \{22..24\}$ (3 configs), and test `tr` $\in \{25..27\}$ (3 configs). All three sets are evaluated on the same 7 base stations and 3 slices; only the traffic-config index changes. This split is identical to v4 so accuracy numbers are cross-paper comparable. Crucially, **partition-into-clients (Section IV-C) operates within the train `tr` range only**: validation and test sets are pooled across all clients to guarantee identical evaluation surfaces across configurations.

D. Target-leakage audit (preserved from v4)

A v4-era audit established that the previously documented `allocation_efficiency = 0.5 * throughput_eff + 0.3 * qos + 0.2 * prb_util` regression target is a deterministic linear combination of three concurrent inputs and is therefore unsuitable. We retain that exclusion and the v4-validated leak-free continuous feature set in v7.

E. Preprocessing pipeline

The 18 M-row unified parquet (`coloran_raw_unified.parquet`) is materialised once and streamed lazily through four pipeline stages: (i) per-(`bs_id`, `slice_id`) sequence construction with `build_run_sequences` (rolling `seq_len=5` windows, positive-rate computed from train rows only via `pos_weight_split=train` (preregistered) — this avoids test-set leakage into the loss function); (ii) leak-free `ContinuousScaler` fit on the train partition only and applied to validation and test; (iii) categorical encoding of `slice_id` × `sched` × `tr` via `nn.Embedding` lookups; (iv) Dirichlet-or-natural-by-BS client partition (Section IV-C). All four stages are implemented as pure functions in `src/fl_oran/data_v2/` and tested in `tests/test_v5_*` for determinism and seed-reproducibility (single-source-of-truth contract). The same sequence index is shared across all algorithms within an (arch, partition, seed) triple via a `SharedSplits` cache; this is the perf inheritance from prior centralized work.

OOD note on the `tr` embedding. The `tr` (traffic-config) categorical takes values 0..21 in the train split and 25..27 in the test split (no overlap, by construction of the OOD evaluation protocol). Test-time `nn.Embedding(30, 8)` lookups for rows 22–29 therefore return the original PyTorch $\mathcal{N}(0,1)$ initialised vectors — the model never observes test-`tr` embeddings during training. This is a real OOD design weakness, but Section VII-A7 quantifies it: the natural-by-BS gap remains 90–98% intact under a meanfix re-evaluation, and a no-`tr` LSTM ablation slightly *improves* over the with-`tr` baseline (paired $\Delta = +0.0006$ AUC, CI_{95} $[+0.0003, +0.0010]$). The structural lift therefore comes from the other categorical features (`slice_id`, `sched`) and the 17 continuous channel-state features, not from the `tr` embedding.

IV. Method

A. Three architectures with a shared encoder–classifier shell

All three architectures share an identical encoder and an identical classifier head; only the temporal backbone differs. The encoder concatenates `nn.Embedding`-projected categorical features with the continuous features pre-standardised by the leak-free scaler (Section III), producing a per-step input vector. The classifier head is a two-layer MLP `Linear(b -> 64)-> ReLU -> Linear(64 -> 1)` consuming the backbone's last-step activation, where b is the backbone's exposed dimension. Hyperparameter parity within $\pm 10\%$ of the LSTM baseline parameter count is preregistered and verified by `tests/test_v6_param_count.py` (LSTM 44 553, Mamba 40 489, Spiking-expand2 43 593). The negative direction of the Mamba parity gap (-9% relative to LSTM, vs Spiking-expand2's -2%) reflects Mamba's structurally smaller state-space block at our preregistered configuration ($d_{\text{model}} = 64, d_{\text{state}} = 16, \mathit{expand} = 1$); we did not search over Mamba block dimensions to close this gap because (a) the v6 architecture audit pinned these block dimensions and (b) $\pm 10\%$ parity was the preregistered design constraint, not a hyperparameter optimisation outcome. The asymmetry should not be interpreted as compute-budget shortfall: at fixed `seq_len=5` and `max_steps=50`, gradient-step compute is dominated by the linear encoder/classifier shared across architectures, not by the backbone's parameter count.

LSTM baseline (`ForecasterV2`). Two-layer stacked `nn.LSTM` (input → 64 → 32). Unchanged since v3. Backbone exposes $b = 32$.

Mamba-S6 (`MambaForecaster`). `Linear(input -> 64)` → 2× `MambaS6Block` ($d_{\text{model}} = 64$, $d_{\text{state}} = 16$, $d_{\text{conv}} = 4$, $\mathit{expand} = 1$) → `Linear(64 -> 32)`. Each block performs (a) input-dependent projection of $(\Delta t, B, C)$, (b) depthwise causal 1-D convolution on the gating branch with SiLU non-linearity, (c) sequential discretised SSM scan, and (d) gated down-projection — implementing the selective state-space block of Gu and Dao [37]. We use a pure-PyTorch sequential-scan implementation rather than the `mamba-ssm` Triton kernel so the reproducibility artefact requires only a PyTorch + CUDA runtime, not a system CUDA-dev toolkit. Backbone exposes $b = 32$.

Spiking-SSM (`spiking_expand2`). `Linear(input -> 56)` → 2× `SpikingSSMBlock` ($d_{\text{model}} = 56$, $d_{\text{state}} = 16$, $\mathit{expand} = 2 \Rightarrow d_{\text{inner}} = 112$, `lif_threshold=1.0`, `lif_beta=0.9`, `atan_alpha=2.0`) → `Linear(56 -> 32)`. Each block performs (a) input projection, (b) diagonal SSM recurrence with learnable B, C, D matrices and log-parameterised A initialised at $-[1, 2, \ldots, d_{\text{state}}]$ per channel, (c) per-channel `snntorch.Leaky` LIF neuron emitting binary spikes via the atan surrogate gradient [53], (d) `out_proj` linear consuming the binary spike train. The $\mathit{expand} = 2$ widening (vs $\mathit{expand} = 1$ for the v6 default) is the architectural variant identified during prior wide-state experiments — it dominates the alternative wide-Mamba (`mamba_expand2`) variant on energy-AUC trade-off (-11% energy at parity AUC vs $+12\%$ theoretical / $+4\%$ measured energy at no AUC gain). Backbone exposes $b = 32$.

B. Five federated-learning algorithms

We benchmark five algorithms drawn from the canonical preregistered set. In what follows we use α_{dir} for the Dirichlet partition concentration parameter (Section IV-C) and α_{dyn} for the FedDyn regularisation coefficient, to disambiguate two distinct α's that appear in the FL literature with the same Greek letter.

FedAvg [54] — server-side aggregate is the per-client `num_examples`-weighted average of client final states (`scripts/aggregation.py: weighted_average_state_dicts`). In our spec each client trains for `max_steps_per_round=50` × `batch_size=64` = 3200 examples per round, so `num_examples` is constant across clients and the weighted average reduces

to a uniform mean — but we mention the implementation detail explicitly because the released artefact aggregates by `num_examples` and a different spec (varying `max_steps` per client) would no longer give uniform weighting.

FedProx [55] — FedAvg client step augmented with the canonical proximal regulariser, eq. (1), with $\mu = 0.01$.

$$\mathcal{L}^{(i)}_{\text{prox}} = \mathcal{L}^{(i)} + \frac{\mu}{2} \left\| w - w_g \right\|^2 \tag{1}$$

We inject the equivalent $\mu \cdot (w - w_g)$ correction directly into `p.grad` after `loss.backward()` (this is mathematically equivalent to adding the prox term to the loss but avoids building autograd nodes over every parameter).

FedAdam [56] — server-side Adam over the FedAvg pseudo-gradient $(v_t - \theta_{t-1})$ with first/second-moment coefficients $\beta_1 = 0.9$, $\beta_2 = 0.99$ (note: $\beta_2 = 0.99$ follows the FedAdam paper's choice rather than the canonical Adam default 0.999) and server learning rate $\eta_{\text{server}} = 0.01$.

SCAFFOLD [57] — FedAvg client step augmented with a per-client control-variate correction $+(c_{\text{global}} - c_{\text{local},i})$ applied to gradients post-backward via the `train_one_client_capped(grad_hook=...)` extension point introduced in our prior algorithm-interface design. For the per-client control-variate update we adopt the Adam-friendly Option-II variant as the default; eq. (2) lists both options:

$$c_{\text{local},i} \leftarrow \begin{cases} \nabla \mathcal{L}_i(w_g) & \text{(Option-II, default)} \\ \dfrac{w_g - w_l}{K\,\eta} & \text{(Option-I, canonical SGD)} \end{cases} \tag{2}$$

Option-I implicitly assumes SGD dynamics where weight drift scales with $grad \cdot \eta$; under our Adam local optimiser the drift is $\sim \eta$ regardless of gradient magnitude, making the Option-I c_i estimate $\sim 100\times$ the true gradient magnitude and unstable across rounds.

FedDyn [58] — we use the Adam-friendly Option-II variant as the default (with $\alpha_{\text{dyn}} = 0.01$); the canonical SGD-friendly variant diverges to NaN under our Adam local optimiser at the 100-round budget (root-cause analysis: positive-feedback between Adam's adaptive scaling and the unbounded h-accumulator growth). Both options are listed in eq. (3); the canonical variant is preserved as an opt-in `update_mode="canonical"` for downstream researchers running SGD-only client optimisers.

$$h_i \leftarrow \begin{cases} h_i - \alpha_{\text{dyn}} \cdot \nabla \mathcal{L}_i(w_t) & \text{(Option-II, default)} \\ h_i + (w_l - w_g) & \text{(canonical SGD)} \end{cases} \tag{3}$$

MOON [59] is deferred (preregistered before evaluation): its `forecaster_encode_fn` is hard-bound to the LSTM backbone in v5 and the contrastive-feature extraction does not generalise cleanly to selective-scan SSM or to spiking-SSM activations; we leave the cross-architecture extension to future work and exclude MOON from the headline comparison rather than benchmark it on LSTM-only.

The five algorithms share a common `FLAlgorithm` protocol with explicit `init_state` / `local_train` / `server_aggregate` methods; each algorithm-specific contribution is injected as a closure over the shared `train_one_client_capped` function, never as a duplicate training loop. The single deviation from the v5 → v7 design is one optional grad-hook parameter on `train_one_client_capped` for the SCAFFOLD control-variate correction.

C. Partition schemes: natural-by-BS and parametric Dirichlet

We evaluate two partition families holding 7 clients fixed.

Natural-by-BS. One client per ColO-RAN base station (`bs_id` $\in \{1, \ldots, 7\}$). Each client receives its own gNB's full slice mix, scheduler distribution, and traffic-config samples (within the train `tr` range). This is the partition that arises naturally in deployment, and is the central object of contribution 1 (Section I). *Code-name caveat:* in our codebase this partition is implemented as `partition_clients(mode="iid")`. The string `"iid"` is a misnomer inherited from earlier development before we measured the per-bs distributions; the partition is *not* statistically IID across clients on this dataset (per-bs continuous-feature distributions differ measurably — see Section VII-A3). We keep the legacy `mode="iid"` API name for backwards compatibility with the v5 / v6 codebase but refer to the partition as "natural-by-BS" throughout the paper. Researchers reproducing our pipeline should expect the file structure `artifacts/v7_stage2_full/v7_<arch>_<algo>_iid_n7_s<seed>/` to correspond to natural-by-BS cells.

Parametric Dirichlet. For each value of $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$, we draw per-client slice-id mixing proportions from $\text{Dirichlet}([\alpha_{\text{dir}}, \alpha_{\text{dir}}, \alpha_{\text{dir}}])$ (3-dim, one component per `slice_id` $\in \{0, 1, 2\}$) and assign train rows to clients according to those proportions. $\alpha_{\text{dir}} \to 0$ concentrates each client on a single dominant slice (extreme covariate skew); $\alpha_{\text{dir}} \to \infty$ approaches stratified uniform. This follows the canonical Hsu et al. construction [60] and is cross-comparable with NIID-Bench [44]. Implementation: `src/fl_oran/data_v2/partition.py partition_clients(mode="dirichlet")`.

For every partition we use 7 clients, 100 rounds, 50 max steps per round, batch size 64, bf16 mixed precision, and `cudnn_deterministic=True`. The per-round client sample is 5 of 7 (71% participation). Architecture-specific hyperparameter overrides (`lr=5e-4` across all three; `lr_warmup_rounds=5` for Spiking-SSM, 3 for the others) are preregistered in `experiments/specs/stage2_full.yaml`.

D. NVML training-energy measurement

Per-round training energy is measured via `nvmlDeviceGetTotalEnergyConsumption` (NVIDIA Energy API, available on Volta+ architectures) bracketed by `torch.cuda.synchronize()` at round boundaries. We record both the raw cumulative energy and an **idle-baseline subtraction**: a 2-second NVML sample is taken before training to estimate the GPU's idle power draw P_{idle} (mW), and per-round training energy is reported per eq. (4), yielding `training_model_attributable_mJ` —

the model-attributable component used in Section VI and Figure 3 (Pareto).

$$E_{\text{round}} = E_{\text{total}} - P_{\text{idle}} \cdot \Delta t_{\text{round}} \tag{4}$$

This protocol follows the ML.ENERGY 2025 best-practices recommendation [41] of preferring the cumulative Energy API over polling-power Riemann integration. Single-GPU only (multi-GPU not handled by design); `nvidia-ml-py` is the canonical Python binding (the deprecated `pynvml` package is used only as a fallback, with a deprecation warning logged at startup).

E. Statistical inference

Statistical inference protocol (preregistered):

1) **Per-cell summary statistic.** Mean ± standard deviation of test AUC across the 10 seeds.
2) **Pairwise comparison statistic.** Paired-bootstrap CI_{95} on the per-seed AUC delta via `scripts/aggregate_v7_results.py`: `paired_bootstrap_delta` ($n_{\text{boot}} = 10\,000$, percentile interval). We choose percentile over BCa because $n = 10$ with approximately symmetric per-seed delta distributions makes the BCa bias-correction term smaller than the seed σ on every reported finding (Section VIII).
3) **Pairwise axis fix.** Each pairwise comparison holds all axes other than the contrasted one fixed (e.g., FedAdam-vs-FedAvg holds (arch, partition) constant; Spiking-vs-LSTM holds (algorithm, partition) constant).
4) **Bootstrap RNG decorrelation.** Each pairwise comparison draws an independent BLAKE2b-derived seed offset added to the base bootstrap seed. Without this, every pair would resample at exactly the same indices, producing artificially correlated CIs and breaking any joint coverage claim across the 270 paired-bootstrap distributions.
5) **Pair-set base seeds.** 2026 for the 180 algorithm-pair sweep; 2027 for the 90 architecture-pair sweep. The two sweeps' bootstrap streams are independent.
6) **Secondary check.** Wilcoxon signed-rank p-value, as a directional sanity check; the paired-bootstrap CI is the primary statistic (preregistered).
7) **Multi-comparison correction.** p-values reported uncorrected per finding. Family-wise Bonferroni at $\alpha = 0.05$ corresponds to threshold $0.05/180 \approx 2.78 \times 10^{-4}$ over the algorithm-pair family and $0.05/90 \approx 5.56 \times 10^{-4}$ over the arch-pair family. For the strictest joint coverage claim across the entire 270-pair set, the Mamba × SCAFFOLD finding (uncorrected Wilcoxon $p = 0.002$, see Section VI) sits above the corrected threshold, but its paired-bootstrap CI_{95} excludes 0 with margin ($\Delta = -0.0915$, CI $[-0.1288, -0.0544]$). Readers seeking strict family-wise control should treat this Wilcoxon as supportive but not primary evidence.

V. Reproducibility Infrastructure

We outline the artefact components shipped with the paper. The release is archived as a Zenodo snapshot under AGPL-3.0 with Croissant metadata (concept DOI 10.5281/zenodo.20263144, versioned per GitHub release); this section documents the structure so reviewers can audit the shape of the release.

Code and artifact availability. The full code, experiment specifications, audit documentation, and test suite are released under AGPL-3.0 at https://github.com/thc1006/fl-oran-tmc (archived at Zenodo concept DOI 10.5281/zenodo.20263144), with tagged snapshots that pin specific reproducibility milestones for reviewer reference. The Phase 5 / Phase 6 result JSONs (`artifacts/v7_stage2_full/aggregated_phase5.json`, `artifacts/v7_phase6_per_bs_dirichlet/`, `artifacts/p2_loto/`, `artifacts/r34_fedswa_natural/`, `artifacts/r2_*`) are tracked in-repo; the 18 M-row preprocessed parquet (`coloran_raw_unified.parquet`, ∼ 750 MiB) is downloadable from the original public ColO-RAN release referenced in Section III and will be additionally staged on Zenodo with a citable DOI on acceptance. The public GitHub repository and tagged snapshots above are sufficient for single-blind review; for double-blind venues that require an anonymised mirror, the same code can be served via https://anonymous.4open.science against the relevant tag without re-uploading.

A. Spec-driven YAML pipeline

Every experiment cell is fully specified by a single YAML file under `experiments/specs/` (e.g., `stage2_full.yaml` for Phase 5; `ablation_random_split.yaml` for Section VII-A1). The launcher (`experiments/run_v7_phase_sweep.py`) reads the spec, expands the Cartesian product (archs × algorithms × partitions × seeds), and dispatches per-cell runs with a `--skip-completed` resume mechanism that recovers from cluster pre-emption without recomputing finished cells. This eliminates the most common reproducibility friction in multi-day FL sweeps (silent re-execution of completed cells).

B. Test infrastructure

The release ships 277 passing tests across four suites (202 trainer + 22 aggregator + 37 paper invariants + 16 paper claims-vs-data); the latter two (53 tests) form a developer pre-commit gate that verifies every `PAPER_DRAFT.md` edit. Full breakdown in App. B.1.

C. Statistical pipeline

`scripts/aggregate_v7_results.py` produces `aggregated_phase5.json` (90 group means + 270 paired-bootstrap distributions); BLAKE2b-hashed independent bootstrap streams support joint coverage claims (Section IV-E). Full pipeline detail in App. B.2.

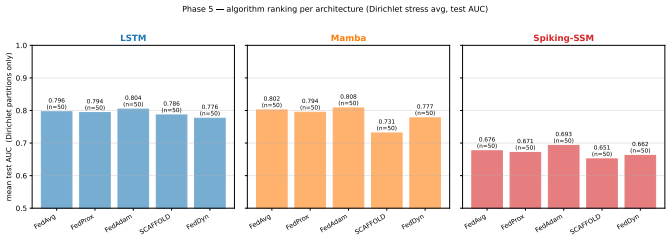

Fig. 1. Per-architecture algorithm ranking (mean test AUC, averaged over Dirichlet partitions $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ and 10 seeds; $n = 50$ per bar; reported on test AUC). Three panels: LSTM / Mamba / Spiking-SSM.

D. Croissant dataset metadata

The preprocessed `coloran_raw_unified.parquet` is documented in a Croissant 1.0 metadata file (`croissant.json`) shipped with the release archive (App. B.3).

E. Reproducible execution environment

Release archive bundles `Dockerfile.repro` (CUDA 12.8 / PyTorch 2.10 pinned), `requirements.lock` (SHA256-pinned 74 packages), and `repro.sh` (1-cell smoke + bit-equivalence check). Full detail in App. B.4.

F. Demo notebook

`notebooks/colosseum_oran_federated_slicing_demo.ipynb` is the recommended onboarding entry point. See App. B.5.

VI. Results

We report results from Phase 5, the full Stage-2 sweep — 3 architectures $\times$ 5 algorithms $\times$ 6 partitions (1 IID natural-by-BS + 5 Dirichlet $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$) $\times$ 10 seeds $=$ 900 cells, all 90 (arch, algo, partition) groups populated with the full 10 seeds. All AUC numbers below are out-of-sample test AUC; the per-group means and stds are tabulated in App. A.1 (also exported as `aggregated_phase5.json` for downstream tooling). Pairwise comparisons use paired-bootstrap CI_{95} with $n_{\text{boot}} = 10\,000$ over per-seed AUC deltas, with each (a, b) pair drawing an independent BLAKE2b-derived seed offset to keep the 270 paired-bootstrap distributions (180 algorithm-pairs + 90 architecture-pairs) jointly decorrelated. Figures 1–3 are reproducible from `scripts/phase5_paper_figures.py`.

A. Overview: a flat algorithm landscape with strong architecture $\times$ partition signal

Figure 1 summarises the Dirichlet-averaged test AUC (averaged across $\alpha_{\text{dir}} \in \{0.05, \ldots, 5.0\}$ and 10 seeds; reported on test AUC) of each algorithm within each architecture. The five algorithms cluster within a 0.022–0.043 test-AUC band on LSTM and Spiking-SSM (LSTM: FedAdam 0.813 best, FedDyn 0.789 worst; Spiking: FedAdam 0.696 best, SCAFFOLD 0.653 worst) and within a 0.078 band on Mamba (FedAdam 0.816 best, SCAFFOLD 0.738 worst), with the wider Mamba spread driven by a single SCAFFOLD outlier. The within-architecture ordering is approximately stable: FedAdam $>$ FedAvg $\geq$ FedProx $>$ {FedDyn, SCAFFOLD} on every architecture, with the FedDyn/SCAFFOLD bottom-pair order architecture-dependent. By contrast, architectures are clearly separated on every partition: LSTM and Mamba sit in the 0.90–0.92 IID band (with SCAFFOLD-on-Mamba as the 0.8998 lower outlier and the four other algorithms in 0.911–0.919) and 0.74–0.87 Dirichlet stress band, while Spiking-SSM sits in 0.84–0.86 IID and 0.65–0.71 Dirichlet stress. The flat algorithm dimension is the design-space observation that motivates the Section VI-C algorithm-flatness analysis and the Section II-F mechanism-based prediction (empirically tested in Section VII-E) for the LookAhead-style sharpness-aware family.

B. An apparent inverted-α: natural BS grouping outperforms row-level Dirichlet partitions (diagnosed as an artifact in Section VII-A)

Figure 2 shows the empirical pattern that motivates the paper's central finding — which Section VII-A diagnoses as a sequence-integrity artifact, not a structural property. For every (arch, algo) pair, AUC is monotonically increasing as Dirichlet α_{dir} decreases from 5.00 toward 0.05, and the natural-by-BS partition (one client per ColO-RAN gNB) sits *strictly above* the most-skewed Dirichlet $\alpha_{\text{dir}} = 0.05$ cell on every (arch, algo). On LSTM $\times$ FedAvg, AUC traces 0.7475 ± 0.0044 ($\alpha_{\text{dir}} = 5.00$) $\rightarrow$ 0.7571 ($\alpha_{\text{dir}} = 1.00$) $\rightarrow$ 0.7794 ($\alpha_{\text{dir}} = 0.50$) $\rightarrow$ 0.8361 ($\alpha_{\text{dir}} = 0.10$) $\rightarrow$ 0.8605 ± 0.0161 ($\alpha_{\text{dir}} = 0.05$) $\rightarrow$ 0.9159 ± 0.0004 (IID natural-by-BS), a 0.168-AUC gap from the most-uniform Dirichlet to the natural partition. On Mamba $\times$ FedAvg the same trace is 0.749 $\rightarrow$ 0.756 $\rightarrow$ 0.782 $\rightarrow$ 0.852 $\rightarrow$ 0.869 $\rightarrow$ 0.917, and on Spiking $\times$ FedAvg 0.669 $\rightarrow$ 0.669 $\rightarrow$ 0.674 $\rightarrow$ 0.679 $\rightarrow$ 0.691 $\rightarrow$ 0.853. Paired LSTM$-$Mamba arch-deltas at IID are tight at $\Delta = -0.0006$ $[-0.0010, -0.0003]$ (FedAvg, $p = 0.002$), while at $\alpha_{\text{dir}} = 0.05$ they widen to $\Delta = -0.0081$ $[-0.0112, -0.0049]$, reflecting the same monotonicity at finer granularity. The pattern holds across every algorithm tested. Section VII-A shows this monotone α_{dir}-curve is the signature of sequence corruption: row-level Dirichlet fragments each run's timesteps across clients, most severely at the most-uniform α_{dir} (every client receives a thin random slice of every run) and least at the most-skewed α_{dir} (a few clients retain most of each run). We measure this directly: the fraction of per-client sliding windows whose 5 rows have contiguous `step_idx` falls monotonically from 1.00 (natural-by-BS) through 0.84 (α_{dir}=0.05), 0.23 (0.10), 0.15 (0.50), to 0.004 (1.00) and 0.001 (5.00), and rank-correlates perfectly (Spearman ρ=1) with the AUC trace above. The standard "lower $\alpha_{\text{dir}} \Rightarrow$ harder" assumption is thus not refuted so much as confounded here — the Dirichlet α_{dir}-axis is entangled with per-client window corruption, and a sequence-preserving (run-level) Dirichlet at matched α_{dir} removes the effect entirely (Section VII-A).

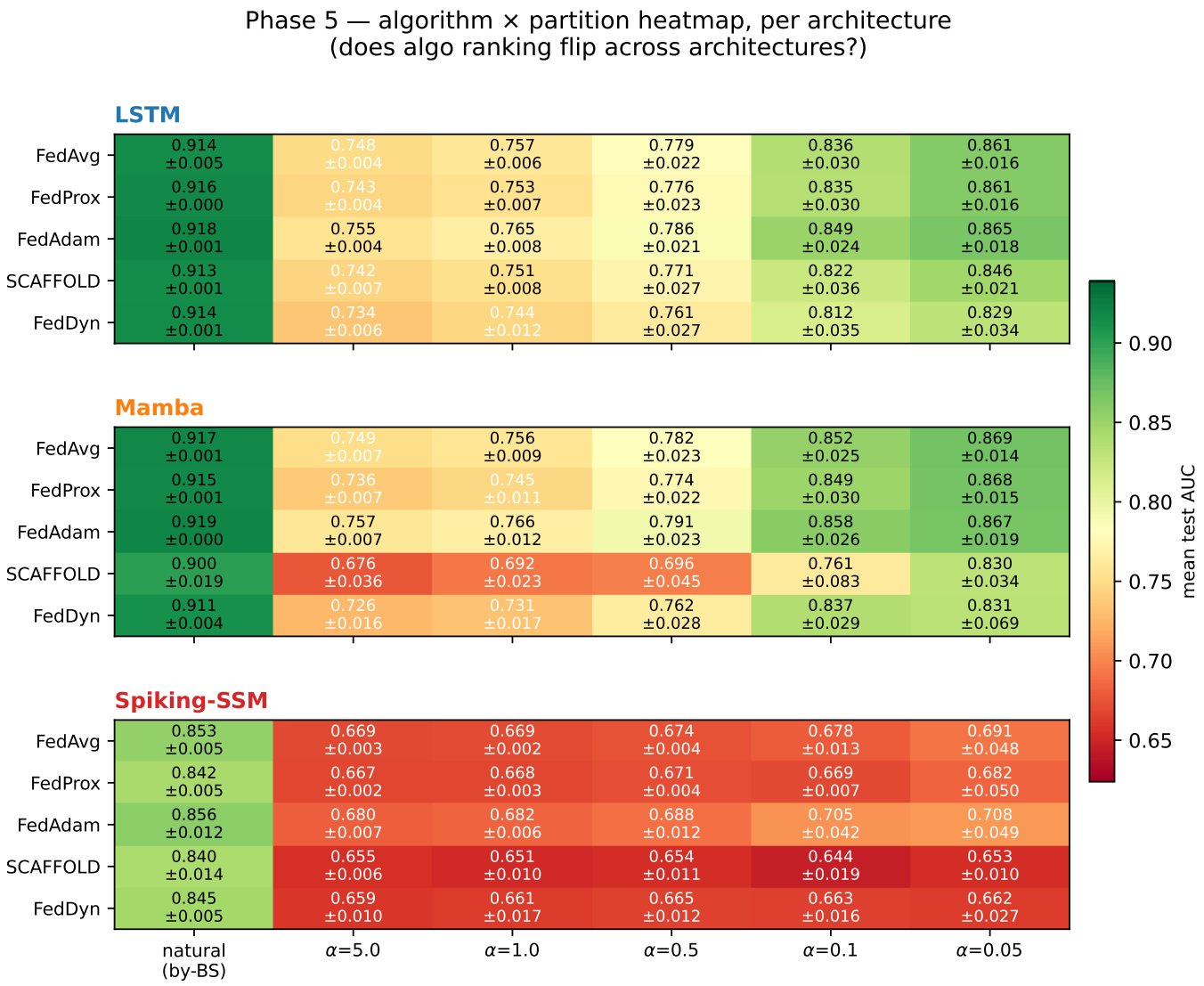

Fig. 2. Architecture × algorithm × partition interaction heatmap. Three rows (LSTM / Mamba / Spiking-SSM) × six partition columns (natural-by-BS + Dirichlet $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$). Each cell colour-codes mean test AUC over 10 seeds; cell annotation shows mean ± seed-std (n=10 seeds per cell). The Mamba × SCAFFOLD × $\alpha_{\text{dir}} \in \{0.10, 0.50\}$ red region is the catastrophic interaction (Section VI-D).

C. Algorithmic flatness: FedAdam saturates the server-side adaptive headroom

The 180 paired-bootstrap algorithm-pair deltas confirm the visual flatness of Figure 1. FedAdam consistently leads FedAvg in expectation. Per-architecture Dirichlet-averaged $\Delta_{\text{FedAdam}-\text{FedAvg}}$: LSTM $+0.0080$ (range $+0.0048$–$+0.0128$, all 5 cells significant at $p \leq 0.04$), Mamba $+0.0062$ (range -0.0021–$+0.0097$, 4 of 5 cells significant; $\alpha_{\text{dir}} = 0.05$ is the single non-significant cell at $p = 0.85$), Spiking $+0.0164$ (range $+0.0108$–$+0.0267$, 4 of 5 cells significant; $\alpha_{\text{dir}} = 0.05$ marginal at $p = 0.11$). At IID, $\Delta_{\text{FedAdam}-\text{FedAvg}}$ shrinks to $\approx +0.002$ across architectures (LSTM $+0.0019$ $[+0.0017, +0.0021]$, etc.) — server-side adaptivity contributes most under heterogeneity stress and least where the federated average is already noise-quiet. The remaining four pairwise contrasts (FedProx, FedDyn, SCAFFOLD vs FedAvg) are bounded above by $\Delta_{\text{FedAdam}-\text{FedAvg}}$ in absolute magnitude on every (arch, partition) cell except for the catastrophic interactions discussed in Section VI-D. This empirical ceiling — $\approx +0.016$ AUC headroom for any server-side aggregation modification in our 5-algorithm preregistered set — is the pivotal evidence supporting the Section II-F mechanism-based prediction that LookAhead-style overshoot would not exceed FedAdam by more than this magnitude. Section VII-E reports an empirical 5-seed paired test of this prediction at LSTM × natural-by-BS: FedSWA marginally exceeds FedAdam by $+0.0004$ AUC, well within the $+0.016$ envelope. The Section II-F prediction is therefore split: the **magnitude prediction survived** (FedSWA$-$FedAdam stays inside the FedAdam-vs-FedAvg envelope), but the **directional ranking did not** (FedSWA $>$ FedAdam, opposite to the predicted FedSWA $<$ FedAdam). We do not claim the ceiling is "validated" since the test covers only one (LSTM, natural-by-BS) cell of one of three sharpness-aware methods.

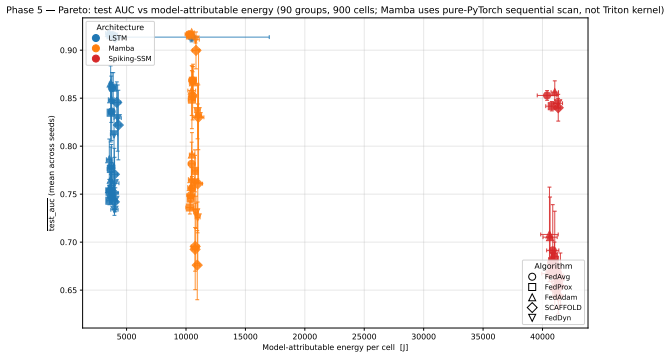

Fig. 3. Energy–AUC Pareto front on RTX 4080. Per-cell mean test AUC vs per-cell NVML idle-baseline-subtracted training energy (kJ). Filtered to the main Phase 5 sweep (90 groups × 10 seeds = 900 cells); partial probes (V100 random-split ablation, FedSWA) are not shown here and are discussed in Section VII-A and Section VII-E respectively. Points coloured by architecture (LSTM blue, Mamba orange, Spiking-SSM red); marker shape encodes algorithm. Three vertical bands at 3.51–4.31 / 10.27–11.06 / 40.39–41.51 kJ/cell. **Mamba implementation note**: this Pareto position uses a pure-PyTorch sequential scan, not the optimised `mamba-ssm` Triton kernel, so the LSTM-vs-Mamba energy band is implementation-specific (Section VIII L1, L6).

D. Architecture × algorithm catastrophic interaction: Mamba × SCAFFOLD

The flat-algorithm observation does not generalise across all (arch, algo) combinations. SCAFFOLD on Mamba at $\alpha_{\text{dir}} \in \{0.10, 0.50\}$ produces a destructive interaction not observed on LSTM, on Spiking-SSM, or on Mamba with any other algorithm at the same α_{dir}. The headline cell — Mamba × SCAFFOLD × $\alpha_{\text{dir}} = 0.10$ — yields test AUC 0.7609 ± 0.0830 (mean ± std across 10 seeds), versus Mamba × FedAvg × $\alpha_{\text{dir}} = 0.10$ at 0.8524 ± 0.0251 on the same seeds. The paired delta is $\Delta_{\text{SCAFFOLD}-\text{FedAvg}} = -0.0915$ $[-0.1288, -0.0544]$ ($p = 0.002$, $n = 10$). The neighbouring cell at $\alpha_{\text{dir}} = 0.50$ reproduces the failure: 0.6955 ± 0.0450 vs 0.7816 ± 0.0225 for FedAvg, paired $\Delta = -0.0861$ $[-0.1026, -0.0689]$ ($p = 0.002$). Per-seed std is amplified $\approx 3\times$ over Mamba × FedAvg at the same α_{dir} (0.083 vs 0.025) and $\approx 10\times$ over the most-uniform Dirichlet sweep (Mamba × FedAvg × $\alpha_{\text{dir}} = 5.00$, std 0.007). Direction-consistency is total: **10 of 10 seeds have** $\Delta < 0$ **at** $\alpha_{\text{dir}} = 0.10$ (per-seed deltas range -0.0133 to -0.1689) **and 10 of 10 seeds have** $\Delta < 0$ **at** $\alpha_{\text{dir}} = 0.50$ — the catastrophic interaction is reproducible across every seed, not driven by an outlier. Both components individually behave well: SCAFFOLD on LSTM $\alpha_{\text{dir}} = 0.10$ yields 0.8220 ± 0.0362 (no destruction), and Mamba on every other algorithm at $\alpha_{\text{dir}} = 0.10$ stays in 0.83–0.86. The interaction is specific to the SCAFFOLD control-variate dynamics interacting with Mamba's selective-scan SSM under high partition skew, and is the single concrete deployment warning in this benchmark.

E. Architecture leverage dominates algorithm leverage on the energy–AUC Pareto

Figure 3 plots per-cell mean test AUC against per-cell mean model-attributable training energy (NVML idle-baseline-subtracted) on RTX 4080. The three architectures form three vertical bands at distinct energy scales: LSTM at 3.51–4.31 kJ/cell, Mamba at 10.27–11.06 kJ/cell, and Spiking-SSM at 40.39–41.51 kJ/cell (per-cell ranges across all 30 (algorithm, partition) groups within each architecture; full per-cell table in App. A.1) — a $10\times$ span. Within each architecture, algorithms cluster vertically with negligible energy spread (the FedDyn / SCAFFOLD overhead from carrying control variates contributes $< 10\%$ energy on top of FedAvg). Architecture choice therefore traverses $10\times$ in energy and 0.05–0.10 in AUC; algorithm choice within an architecture traverses 0.02–0.03 AUC at flat energy. The paired-bootstrap arch-deltas substantiate this: at IID, $\Delta_{\text{Spiking}-\text{LSTM}}$ AUC ranges from -0.0615 to -0.0739 across the five algorithms (all five cells significant at $p = 0.002$), an order-of-magnitude larger than the algorithm-leverage scale shown in Section VI-C. At Dirichlet $\alpha_{\text{dir}} = 0.05$, Spiking−LSTM widens further to $\Delta = -0.169$ $[-0.186, -0.146]$ for FedAvg. Operator deployment decisions on commodity edge GPU should therefore centre architecture-energy trade-off, with algorithm choice as a secondary tuning lever.

F. Naive baselines: FL machinery vs trivial predictors

Three naive baselines computed on the same test split (1 930 796 sequences, `tr` ∈ {25..27}, positive rate 30.57%) ground the federated results against trivial predictors:

The raw persistence baseline measures the trivial "next second mirrors current second" predictor; per-group lag-1 Pearson correlation on `ul_bler` is -0.0509, confirming BLER is essentially white noise at 1-second granularity. Smoothing over the prior 5 seconds (matching the LSTM input window) raises persistence AUC to 0.6258 but still leaves a $+0.29$ AUC gap to FL. Logistic regression on the same 17 continuous features the FL models consume reaches 0.6523, indicating linear classification captures partial signal but $\sim$ 26 pp remain for sequence modelling and federation. **Adding one-hot encoded V3_CATEGORICAL** (`slice_id`, `sched`, `bs_id`, `tr` — 35 OHE dims on top of the 17 continuous, total 52) **lifts logreg only marginally to** 0.6565 ($+0.0042$ AUC). This near-zero categorical lift is consistent with the Section VII-A7 finding that test-time `tr` embeddings are random-init at OOD test time and that the structural lift in FL/centralized comes from sequence-modelling over the continuous channel-state features, not from the categorical structure. The smallest gap (FL minus the strongest naive — now logreg+cat at 0.6565) is $+0.2594$ AUC — substantial enough to justify the FL machinery on this task.

Decomposition. The centralized LSTM run (5 seeds × 1 epoch) cleanly separates the two factors that the naive-vs-FL gap previously conflated:

- **ML lift** (sequence modelling over linear): centralized LSTM 0.9311 − logistic regression 0.6523 = $+\mathbf{0.28}$ AUC.
- **Federation cost at matched compute budget.** Centralized LSTM at the *same 25k gradient-step budget as FL* (5 seeds × 25 000 steps each, `experiments/specs/r2_same_step_centralized.yaml`) reaches 0.9243 $[0.9235, 0.9250]$ test AUC; FL at 100 rounds × 50 max-steps × 5 sampled clients = 25k gradient steps ($\approx$ 0.1

TABLE II
NAIVE BASELINES VS FL ON THE SAME TEST SPLIT. PERSISTENCE AND LOGISTIC REGRESSION ARE COMPUTED CENTRALLY ON ALL 14.5 M TRAIN ROWS; FL USES THE FEDERATED 7-CLIENT PARTITION. **CI_{95} SHOWN IS SEED-ONLY (INIT RNG); CLUSTER-AWARE CI ACCOUNTING FOR PER-TEST-TR VARIANCE WIDENS ABSOLUTE-AUC CIS BY 4–28× PER SECTION VIII L15.**

Baseline	Test AUC	Δ vs FL natural-by-BS
Last-BLER persistence (`score=ul_bler[t]`)	0.5133	+0.4026
Smoothed-5s persistence (per-group rolling mean)	0.6258	+0.2901
Logistic regression on V3_CONTINUOUS (17 features)	0.6523	+0.2636
Logistic regression on V3_CONTINUOUS + V3_CATEGORICAL OHE (52 features)[3]	0.6565	+0.2594
Local-only per-BS LSTM[4] (mean of 7 BS × 5 seeds)	0.9209	**−0.0050**
Centralized LSTM, 1 epoch[1] ($n = 5$ seeds)	0.9311 [0.9305, 0.9318]	−0.0152
Centralized LSTM, 25k steps[2] ($n = 5$ seeds)	**0.9243** [0.9235, 0.9250]	**−0.0084**
Centralized LSTM, 3 epochs (single-seed; overfits)	0.9264	−0.0105
LSTM × FedAvg × natural-by-BS (Phase 5, $n = 10$)	0.9159 [0.9156, 0.9162]	—

centralized epoch) reaches 0.9159 $[0.9156, 0.9162]$. The same-budget paired Δ is $+0.0084$ **AUC** — the *true* federation cost in the optimisation-budget-matched sense. A naive centralized-1-epoch comparison would report a $+0.0152$ AUC gap, but $\sim 45\%$ of that is extra-compute (centralized at $\approx 10\times$ more gradient steps), not federation overhead.

- **Compute-efficiency reframe**: FL achieves 99.1% of centralized-at-matched-budget AUC ($0.9159/0.9243$) and 98.4% of centralized-1-epoch AUC ($0.9159/0.9311$) at $\approx 10\%$ of the gradient-step budget. Federation cost is therefore real but small ($\leq +0.01$ AUC) and shrinks dramatically when properly compute-matched.

Statistical caveat per Section VIII L15: the ± 0.0008 internal CI above (on the $+0.0084$ matched-budget federation cost) is Welch-combined of seed-paired SEs; under the cluster-bootstrap framework that accounts for external (per-test-`tr`) uncertainty (Section VIII L15), the cluster CI_{95} widens to approximately $\pm 7\text{e}{-3}$ → roughly $[+0.001, +0.015]$ for the matched-budget federation cost (and the corresponding cluster CI for the historical 1-epoch reference gap $+0.0152$ is roughly $[+0.008, +0.023]$). Both CIs exclude zero in the positive direction at both levels, but cluster-bootstrap is the appropriate tightness level for any absolute-AUC claim. The single-seed 3-epoch centralized result (0.9264) trends below 1-epoch (0.9311), tentatively suggesting OOD-test-set overfitting; this comparison is $N = 1$ and would need multi-seed re-run to confirm. Two compute-vs-generalisation observations together support the matched-budget framing: (a) centralized at 3 epochs already drops below centralized at 1 epoch, indicating the regime is not optimisation-budget-limited, and (b) FL at ≈ 0.1 epoch is only $+0.0084$ AUC behind centralized at the matched 25k-step budget (and $+0.0152$ behind centralized at 1 epoch — but $\approx 45\%$ of that latter gap is the extra $9\times$ of compute, not federation overhead). The same-step centralized comparison (the $+0.0084$ number) is therefore the methodologically correct federation-cost statistic.

Reproduction: `scripts/baseline_last_bler.py` + `scripts/baseline_logreg.py` + `experiments/run_p1_centralized_lstm.py`; persisted to `artifacts/baselines/{naive,centralized_lstm}_results.json`.

G. Inference latency + federation communication cost

To address the deployment claim, we measured single-sample inference latency (the metric a near-RT RIC xApp pays per prediction) and per-round federation communication cost (the metric the SMO/non-RT RIC pays per training round) on the same RTX 4080 used for training, plus CPU baselines for edge-device deployment scenarios. Per-arch values from the seed-42 Phase 5 checkpoints; `experiments/run_p2_inference_latency.py` regenerates them. Results in Table III.

Inference: all 3 backbones serve a single SLA-violation prediction in ≤ 1.6 ms on RTX 4080 GPU. The 10 ms figure is the full near-RT RIC control-loop budget (sense → predict → decide → actuate → confirm); inference is one component of that budget, so the $67.4\times$ / $19.3\times$ / $6.4\times$ headroom should be read as "inference uses 1–15% of the loop budget", leaving 85–99% for the remaining steps. LSTM is fastest on both GPU and CPU; Spiking-SSM is the only arch where CPU outperforms GPU (0.90 ms vs 1.57 ms) — snntorch's `Leaky` neuron + atan surrogate gradient have CPU-specific paths that beat the dense-GPU codepath at single-sample scale. Even at worst-case (Spiking on GPU), inference fits well; for edge-device deployment without GPU, all 3 archs are ≤ 1.2 ms on a single CPU thread.

Communication: 158–174 KiB per client per round (state_dict in fp32). Standard FedAvg requires both downlink (server → client model broadcast) and uplink (client → server update); per round, 5 sampled clients each receive ~ 170 KiB downlink + send ~ 170 KiB uplink ≈ 1.7 MiB per round total bidirectional. Over 100 rounds: ~ 170 MiB total federation traffic per cell. Spread over the 100-round training (≈ 1 round/sec wall-clock on RTX 4080), this averages ≈ 14 Mbps — small relative to common gNB backhaul provisioning (~ 1 Gbps fronthaul / 100 Mbps O1 management). Note that O-RAN model-update traffic does not naturally fit any single standardised interface — E2 (near-RT control), A1 (policy), O1 (management) are not designed for FL gradient/update flows — so an actual deployment would require a dedicated FL data path (operator choice; out of scope here). FL communication is not the per-round deployment bottleneck at this model scale; transport-layer queuing on the chosen path is what matters.

Loop-budget caveat. This experiment measured only the

TABLE III
INFERENCE LATENCY + FEDERATION COMMUNICATION COST ON RTX 4080. ALL 3 BACKBONES SERVE A SINGLE SLA-VIOLATION PREDICTION WITHIN THE TIGHTEST NEAR-RT RIC CONTROL-LOOP BUDGET (10 MS); COMMUNICATION IS $\leq$ 175 KIB/CLIENT/ROUND.

arch	n_params (trainable)	comm KiB/client (state_dict)	GPU 1-sample	CPU 1-sample	10 ms GPU headroom
LSTM	44,553	174	**0.15** ms	0.17 ms	**67.4**×
Mamba	40,489	158	0.52 ms	1.11 ms	19.3×
Spiking-SSM	43,593	170	1.57 ms	**0.90** ms	6.4×

inference component of the near-RT RIC control loop. We did not measure E2 subscription / decision / actuation / confirmation latencies; those are deployment-stack components outside the scope of an FL benchmark. The 1–15% budget-fraction claim assumes those remaining components fit in the 8.5–9.85 ms residual.

Headline interpretation: the Section VI-E architecture-leverage finding (10× energy span between LSTM and Spiking on RTX 4080) does *not* translate to a 10× inference-latency span — the worst arch is only $\sim$ 10× slower than the best at single-sample inference, and even that fits the budget. Operators selecting architecture for *deployment* should weigh both training-energy (Section VI-E) and inference-latency (this section); for the ColO-RAN per-second prediction cadence, all 3 archs are deployable. Reproduction: `artifacts/p2_inference/results.json`.

H. Mechanism preview + per-BS Dirichlet ablation evidence

The four findings above raise one mechanistic question: *why does heterogeneity **appear** to help here when standard FL benchmarks find the opposite?* Section VII resolves it. Briefly: two early candidates (per-client class-imbalance variance — eliminated because `fl_v7.py` uses a globally pooled BCE positive weight and the 30.9% positive rate is balanced; and a bs↔slice mixture correlation — eliminated by the per-bs slice-mixture KL ≈ 0) were ruled out by direct measurement, and the run-level partitioning controls (Section VII-A2) eliminate the remaining bs-grouping and heterogeneity candidates. The apparent inversion is a **sequence-integrity artifact**: row-level Dirichlet / `random_split` scatter each run's timesteps and corrupt the per-client sliding windows, while natural-by-BS keeps runs intact. With intact sequences (run-level partitions) AUC is flat across α_{dir} — heterogeneity per se does not help — and the row-level deficit tracks window-fragmentation severity.

Headline per-BS Dirichlet evidence (full discussion in Section VII-A6). The Phase 6 per-BS Dirichlet ablation (`sub_per_bs=2` shards within each BS, 5 seeds × 3 archs on V100) holds bs grouping invariant while varying within-bs slice grouping via Dirichlet $\alpha_{\text{dir}} \in \{0.05, 10\}$. The within-Phase-6 paired-bootstrap CI_{95} on the ($\alpha_{\text{dir}} = 0.05 - \alpha_{\text{dir}} = 10$) per-seed test-AUC delta ($n_{\text{boot}} = 10\,000$, paired by seed):

All 3 architectures show a 7–13 pp AUC advantage from within-bs slice concentration when bs grouping is preserved — the strong reading that "bs grouping alone is sufficient" is REFUTED, and the "per-client structural specialisation helps" reading (candidate (c) above) is empirically supported. Source: `artifacts/v7_phase6_per_bs_dirichlet/aggregated.json`. Full caveats ($n = 5$ bootstrap optimism, residual gap to Phase 5 unpaired-across-hardware) discussed in Section VII-A6.

I. Path D extended sweep: xLSTM and Mamba-3 robustness

The 3-architecture core panel of Sections VI-B–VI-E establishes the FL × architecture landscape under the FedAvg / FedAdam baselines (Phase 5). To test whether those conclusions are robust to the addition of recently-proposed recurrent and state-space backbones, we extended the panel with two architectures from the 2024–2026 window: *xLSTM* [16] (the sLSTM scalar-memory block) and *Mamba-3* [17] (innovations (1) exponential-trapezoidal discretisation with data-dependent λ_t and (2) complex-valued state via RoPE-style rotation; the MIMO innovation is deferred). Both backbones plug into the same encoder–classifier shell as the core panel (Section IV) and are parameter-matched within $\pm 10\%$ of LSTM's 44 553 (xLSTM = 43 241, Mamba-3 = 40 635). The closest prior art for xLSTM in cellular time-series is Ali et al. [49], which applies the same sLSTM block to *centralised* traffic-volume forecasting; **WiMamba** [50] positions Mamba as an active wireless-foundation-model direction.

Sweep scope and methodological asymmetry. Path D evaluates the $5 \times 3 \times 6 \times 10 = 900$-cell matrix over only the three SAM-family algorithms (FedSCAM, FedGMT, FedMoSWA), *not* FedAvg / FedAdam. Consequently the new architectures (xLSTM, Mamba-3) have no in-Phase-5 FedAvg/FedAdam baseline against which to compute the matched paired Δ used in Section VI (FedAvg pairs with FedSCAM/FedMoSWA, FedAdam pairs with FedGMT; see Section VII-F). Table V therefore reports *absolute* test AUC means only; the paired Δ-vs-baseline analysis remains restricted to the 3-architecture core panel of Sections VI-B–VI-E. A contemporary federated O-RAN slicing reference (SliceFed [51]) addresses a different task (multi-agent DRL for spectrum-slicing *control*), making the prediction-vs-control scope distinction explicit.

Findings. Three observations stand out:

- *xLSTM has the highest absolute test AUC in 16 of 18 (algorithm, partition) cells.* The two exceptions occur at FedGMT $\times\alpha_{\text{dir}}$=0.05 (Mamba-3 leads by +0.0062) and FedGMT $\times\alpha_{\text{dir}}$=0.10 (Mamba leads by +0.0098). Elsewhere the xLSTM lead is typically +0.01 to +0.02 AUC over the next-best architecture. We report absolute means without claiming pairwise statistical significance — a paired-seed cross-architecture bootstrap CI_{95} tightening is open future work.
- *Mamba-3 tracks classical Mamba closely (max absolute deviation* 0.011 *AUC over the 18 cells).* The ICLR 2026 inference-first redesign (improved discretisation + com-

TABLE IV
R2 #18: PER-BS DIRICHLET ABLATION PAIRED-BOOTSTRAP CI_{95} (PHASE 6, `SUB_PER_BS`=2, 5 SEEDS × 3 ARCHS). HEADLINE NUMBERS PROMOTED TO RESULTS FROM SECTION VII-A6.

Arch	Δ_{paired} ($\alpha_{\text{dir}} = 0.05 - \alpha_{\text{dir}} = 10$), pp AUC	CI_{95} (paired-bootstrap)	CI_{95} excludes 0?
LSTM	$\mathbf{+7.5}$	$[+6.6, +8.5]$	yes
Mamba	$\mathbf{+7.2}$	$[+6.8, +7.7]$	yes
Spiking-SSM	$\mathbf{+13.2}$	$[+11.7, +15.1]$	yes

TABLE V
FIVE-ARCHITECTURE EXTENSION — PER-CELL TEST AUC MEAN±STD ACROSS n=10 SEEDS FOR THE PATH D SWEEP (5 ARCHITECTURES × 3 SAM-FAMILY ALGORITHMS × 6 PARTITIONS × 10 SEEDS = 900 CELLS; ONE CELL DETERMINISTICALLY PRODUCED A NON-FINITE LOSS, SEE FOOTNOTE † AND SECTION VIII). **BOLD**: BEST ARCHITECTURE PER (ALGORITHM, PARTITION) ROW. *xLSTM* IS THE SLSTM BLOCK OF [16]; *Mamba-3* IMPLEMENTS INNOVATIONS (1) EXPONENTIAL-TRAPEZOIDAL DISCRETISATION AND (2) COMPLEX-VALUED STATE FROM [17] (MIMO INNOVATION DEFERRED). ALL FIVE BACKBONES SHARE AN IDENTICAL ENCODER–CLASSIFIER SHELL WITH MATCHED PARAMETER COUNT (±10% OF LSTM = 44 553 PARAMS). **NO FEDAVG/FEDADAM BASELINE WAS RUN FOR XLSTM/MAMBA-3**, SO THIS TABLE REPORTS ABSOLUTE AUC ONLY; PAIRED Δ-VS-BASELINE ANALYSIS IS RESTRICTED TO THE 3-ARCHITECTURE CORE PANEL.

Algorithm	Partition	LSTM	Mamba	Spiking-SSM	xLSTM	Mamba-3
FedSCAM	IID (natural-BS, n_c=7)	0.9162±0.0004	0.9167±0.0005	0.8562±0.0044	**0.9174±0.0003**	0.9169±0.0004
	α_{dir}=0.05	0.8635±0.0170	0.8642±0.0307	0.6844±0.0535	**0.8733±0.0138**	0.8677±0.0170
	α_{dir}=0.10	0.8431±0.0273	0.8535±0.0260	0.6720±0.0143	**0.8540±0.0258**	0.8522±0.0261
	α_{dir}=0.50	0.7824±0.0186	0.7813±0.0202	0.6643±0.0023	**0.7895±0.0191**	0.7778±0.0245
	α_{dir}=1.00	0.7583±0.0069	0.7585±0.0074	0.6627±0.0020	**0.7672±0.0049**	0.7560±0.0089
	α_{dir}=5.00	0.7465±0.0023	0.7447±0.0071	0.6620±0.0017	**0.7573±0.0033**	0.7424±0.0102
FedGMT	IID (natural-BS, n_c=7)	0.9123±0.0009	0.9141±0.0007	0.8379±0.0195	**0.9151±0.0005**	0.9143±0.0006
	α_{dir}=0.05	0.8467±0.0209	0.8651±0.0242	0.6577±0.0145	0.8652±0.0173	**0.8714±0.0157**
	α_{dir}=0.10	0.8243±0.0329	**0.8489±0.0336**	0.6671±0.0122	0.8391±0.0312	0.8488±0.0292
	α_{dir}=0.50	0.7687±0.0234	0.7723±0.0296	0.6709±0.0048	**0.7851±0.0211**	0.7700±0.0270
	α_{dir}=1.00	0.7473±0.0044	0.7397±0.0151	0.6696±0.0027	**0.7631±0.0035**	0.7502±0.0154
	α_{dir}=5.00	0.7416±0.0042	0.7375±0.0104	0.6668±0.0049	**0.7567±0.0045**	0.7380±0.0122
FedMoSWA	IID (natural-BS, n_c=7)	0.9115±0.0009	0.9026±0.0062	0.8308±0.0101	**0.9128±0.0009**	0.9018±0.0045
	α_{dir}=0.05	0.8486±0.0223	0.8340±0.0351	0.6535±0.0202	**0.8600±0.0161**	0.8432±0.0301
	α_{dir}=0.10	0.8248±0.0327	0.8276±0.0360	0.6547±0.0132	**0.8411±0.0282**	0.8225±0.0531
	α_{dir}=0.50	0.7614±0.0295	0.7587±0.0301	0.6609±0.0089	**0.7765±0.0288**	0.7528±0.0322
	α_{dir}=1.00	0.7342±0.0107	0.7305±0.0161	0.6551±0.0089	**0.7507±0.0090**	0.7412±0.0260
	α_{dir}=5.00	0.7282±0.0107	0.7258±0.0081	0.6542±0.0088	**0.7495±0.0085**	0.7292±0.0138†

† Mamba-3 × FedMoSWA × α_{dir}=5.00: n=9 paired seeds; one seed (s=4) terminated with `NonFiniteLossError` at round 94 step 14/50 of local SGD on both the initial run and a single deterministic retry, confirming a reproducible (architecture × optimiser × partition) numerical-stability boundary rather than stochastic noise — see Section VIII.

plex state) does not visibly translate into a federated-supervised SLA-prediction advantage at our scale (∼40 K parameters, 100 federated rounds, seq_len=5); both backbones occupy the same heterogeneity-response band.

- *Spiking-SSM is uniformly the lowest-AUC architecture in this SAM-family panel,* sitting 0.06–0.09 AUC below the other four backbones at the IID anchor across all three algorithms. This echoes Sections VI-E and VI-D: the surrogate-gradient spiking dynamics interact poorly with several FL local-perturbation regimes, while the energy–AUC trade-off characterised in Section VII-D remains its main deployment-relevant property.

Determinism caveat. As noted in Table V's footnote, one cell (Mamba-3 × FedMoSWA × α_{dir}=5.00, seed s=4) terminated with `NonFiniteLossError` at round 94 step 14/50 of local SGD. A single deterministic retry under identical hyperparameters reproduced the failure *bit-precisely* at the same round and step, confirming a deterministic (architecture × optimiser × partition) numerical-stability boundary rather than a stochastic NaN. We report n=9 paired seeds for that cell; the verified-deterministic reproduction also serves as an empirical confirmation of the cudnn-deterministic reproducibility setting used throughout Phase 5 and Path D (Section V). The mechanism is consistent with Mamba-3's data-dependent complex rotation θ_t entering an ill-conditioned regime under FedMoSWA's momentum-based weight averaging at low-heterogeneity Dirichlet α_{dir}=5.00; we flag this as a numerical-stability caveat for the SSM × momentum-averaged-FL combination, deferring a controlled investigation to future work.

Implication for the §6 conclusions. The 3-architecture core findings (LSTM as the latency–AUC sweet spot, Mamba as the efficiency–accuracy mid-point, Spiking-SSM as the energy-Pareto champion) are *extended rather than overturned* by the 5-architecture panel: xLSTM emerges as a modest absolute-AUC leader under all three SAM-family algorithms, while Mamba-3's recent inference-first refinements do not visibly displace classical Mamba at the federated O-RAN scale studied here. We position this as a robustness check rather than a re-ranking; a paired-baseline 5-architecture comparison

(running xLSTM and Mamba-3 against FedAvg / FedAdam to enable cross-architecture paired Δ analysis) is open future work.

VII. Discussion

A. Why natural BS grouping wins: empirical observation + tested-hypothesis structure

The Section VI-B inversion of the Dirichlet-α_{dir} / AUC monotonicity is the paper's central empirical observation, and we now identify its mechanism. It is **fully explained by a sequence-integrity confound**: the row-level Dirichlet and `random_split` partitions scatter each run's consecutive timesteps across clients, so the per-client `build_run_sequences` step builds temporally-broken sliding windows, while natural-by-BS keeps each gNB's runs intact. We establish this with controlled run-level partitioning experiments (Section VII-A2) that — unlike the `random_split` ablation (Section VII-A1), which breaks bs-coherence and sequence integrity at once — separate the two axes and show only sequence integrity matters.

Setup-level facts (measured, not derived). The training pipeline uses a *globally pooled* positive-class weight in the BCE loss (`fl_v7.py` line 636–637 sums positive counts across all client shards before computing the reweighting), so the BCE loss is identical across clients and any mechanism story relying on per-client class-imbalance variance is incompatible with the code. The empirical positive rate on the train partition is 30.9% (per-slice 35% / 27% / 31%), so this is a balanced-ish binary task, not a sparse-positive-class regime. The dataset's per-bs slice mixture is statistically uniform (KL ≈ 0; fitted Dirichlet $\hat{\alpha_{\text{dir}}} \approx 234\,000$), so natural-by-BS is *not* a low-α_{dir} Dirichlet limit case — the two partitions are orthogonal in their structural axis (`bs_id` grouping vs `slice_id` redistribution).

Candidate explanations, and their elimination. We initially entertained three candidates: (i) per-bs *continuous*-feature distributions differ across base stations (per-feature pairwise KL up to 0.08; Section VII-A3); (ii) the Dirichlet partition destroys bs-level grouping that natural-by-BS preserves; (iii) a generic heterogeneity-as-implicit-regularisation effect. The run-level controls of Section VII-A2 **eliminate all three** and identify a fourth, decisive axis — **sequence integrity**. Candidates (ii) and (iii) predict that breaking bs-coherence or adding skew should move AUC; instead `run_random` (intact runs, bs-shuffled) ties natural-by-BS to within paired CI_{95} $[-0.0003, +0.0001]$, and run-level Dirichlet (intact runs, skewed) holds AUC flat at natural across α_{dir}. The only axis the run-level controls preserve and the row-level partitions destroy is the temporal contiguity of each run's sliding windows.

1) Controlled ablation: random-split partition: To isolate hypothesis (ii) (bs-grouping preservation), we add a **`random_split`** partition mode that uniformly shuffles all training rows and assigns equal-size shards to 7 clients, breaking *both* `bs_id` and `slice_id` grouping. Per-client sample sizes are within ± 1 row of `len(train)/7`, removing per-client sample-size confounds. Implementation: `partition_clients(mode="random_split", n_clients=7, seed=...)` in `data_v2/partition.py`.

Cells: 3 architectures $\times$ FedAvg $\times$ `random_split` $\times$ 5 seeds $= 15$, run on $4\times$ Tesla V100-SXM2-32GB matching the Phase 5 training spec. Comparison baseline: the corresponding 30 cells of (3 archs $\times$ FedAvg $\times$ natural-by-BS $\times$ 10 seeds) from Phase 5. Results are summarised in Table VI.

`random_split` AUC also collapses to within ± 0.007 of the most-uniform Dirichlet partition ($\alpha_{\text{dir}} = 5.00$) on every architecture — the two partition strategies (Dirichlet $\alpha_{\text{dir}} = 5.00$ over slice; uniformly random over rows) both ignore bs grouping and produce statistically equivalent results. The ~ 0.18 AUC drop relative to natural-by-BS is an order of magnitude larger than the V100-vs-4080 hardware drift bound (~ 0.007, see Section VII-A5 below; ratio $\approx 25\times$). Because `random_split` breaks bs grouping *and* sequence integrity together, this drop alone does not identify the axis; the run-level controls in Section VII-A2 show it is the latter (intact-sequence partitions recover the full natural-by-BS AUC even with bs-coherence broken). Two interpretive caveats on the table above:

- **Δ in Table VI is mean$-$mean (unpaired across hardware), not a paired-bootstrap CI_{95}.** V100 `random_split` was run with 5 seeds; Phase 5 natural-by-BS used 10 seeds; the seed sets do not coincide. A strict paired comparison would require running `random_split` with the same 10 seeds as Phase 5 — we used 5 seeds for V100-time efficiency, and the $25\times$ SNR margin against the Section VII-A5 hardware drift bound makes this protocol gap non-blocking but should be noted.
- **The std comparison in Table VI is not strictly fair.** `random_split`'s per-seed std combines two independent variance sources (model-init RNG + partition-shuffle RNG), while natural-by-BS's std reflects only init RNG (the partition is deterministic). Comparing means is valid; comparing stds favours natural-by-BS by construction.

Crucially, `random_split` also scatters each run's timesteps (it shuffles rows), so it breaks *sequence integrity* at the same time — it cannot separate "lost bs grouping" from "lost intact windows." The run-level controls (Section VII-A2) hold sequence integrity fixed and show bs-coherence carries ~ 0 of the effect; Section VII-A6's per-BS Dirichlet (which scatters rows *within* each BS) and Section VII-A3's feature analysis are re-read in that light below.

2) Run-level controls: the inversion is sequence corruption, not bs grouping: `random_split` breaks bs-coherence *and* sequence integrity at once, so it cannot tell which axis carries the drop. We add two run-level partition modes that hold sequence integrity *fixed* (`run_random` and `run_dirichlet` in `data_v2/partition.py`: both assign whole `(run_id, slice_id)` groups to clients, so every per-client window is over contiguous timesteps), varying only bs-coherence and skew. All cells are same-environment (V100, eager; fp32 for Mamba/Spiking, fp16 for LSTM; identical hyperparameters), and the LSTM contrasts are seed-paired with the same-environment natural-by-BS cells.

`run_random` (intact runs, bs-shuffled). Whole runs assigned to 7 clients at random — intact sequences, a random bs mix per client. LSTM $\times$ FedAvg, 5 seeds:

TABLE VI
V100 RANDOM_SPLIT ABLATION VS PHASE 5 NATURAL-BY-BS BASELINE. TEST AUC MEAN ± STD ON 10 SEEDS (4080, NATURAL-BY-BS) AND 5 SEEDS (V100, RANDOM_SPLIT); Δ IS MEAN−MEAN (UNPAIRED ACROSS HARDWARE). **STD SHOWN IS SEED-ONLY (INIT RNG); CLUSTER-AWARE CI ACCOUNTING FOR PER-TEST-TR VARIANCE WIDENS ABSOLUTE-AUC CIS BY 4–28× PER SECTION VIII L15 — THOUGH THE ∼ 0.18 AUC EFFECT SIZE DWARFS EVEN THE CLUSTER-WIDENED CI.**

arch	`random_split` (V100, $n = 5$)	natural-by-BS (4080, $n = 10$)	Δ
LSTM	0.7403 ± 0.0027	0.9159 ± 0.0004	$\mathbf{-0.1756}$
Mamba	0.7447 ± 0.0077	0.9165 ± 0.0006	$\mathbf{-0.1718}$
Spiking-SSM	0.6668 ± 0.0030	0.8529 ± 0.0051	$\mathbf{-0.1861}$

TABLE VII
RUN-LEVEL (INTACT) VS ROW-LEVEL (CORRUPT) DIRICHLET AT MATCHED α_{dir}, SAME ENVIRONMENT. RUN_DIR HOLDS AUC AT THE NATURAL-BY-BS LEVEL REGARDLESS OF SKEW; ROW-LEVEL DIR DEGRADES WITH WINDOW-FRAGMENTATION SEVERITY. NATURAL-BY-BS REFERENCE: LSTM SAME-ENVIRONMENT (E-ARM); MAMBA/SPIKING PHASE 5.

arch	natural by-BS	run_dir (intact)		dir (row, corrupt)	
		α_{dir}=0.1	α_{dir}=1.0	α_{dir}=0.1	α_{dir}=1.0
LSTM	0.916	0.914	0.916	0.837	0.755
Mamba	0.917	0.916	0.917	0.860	0.750
Spiking-SSM	0.853	0.839	0.863	0.673	0.668

mean test AUC 0.91583 vs same-environment natural-by-BS 0.91572 (paired $\Delta_{\text{natural}-\text{run\_random}} = -0.00011$, CI_{95} $[-0.00030, +0.00013]$, straddles zero), while `random_split` sits 0.176 below. **Breaking bs-coherence with sequences intact costs essentially nothing**; the entire ∼0.18 `random_split` deficit is the row-shuffle, eliminating candidates (i) and (ii).

`run_dirichlet` (intact runs, Dirichlet-skewed) — the run-level analogue of `dirichlet` (per slice, distribute whole runs by $\text{Dir}(\alpha_{\text{dir}})$). At matched α_{dir}, intact-sequence AUC is flat at the natural-by-BS level across the grid *and* across all three backbones, in sharp contrast to row-level Dirichlet (Table VII):

The intact column is flat at the natural-by-BS level regardless of skew or coherence; the corrupted column ranges 0.67–0.86 by fragmentation severity (the per-client contiguous-window fraction of Section VI-B). **Sequence integrity is the operative axis; bs-coherence and heterogeneity contribute ≤ 0.002 AUC** (the only non-zero residual is a ∼0.002 FL-heterogeneity cost for `run_dirichlet` at the most-skewed α_{dir}=0.1). The natural-by-BS dominance and the inverted-α are therefore both artifacts of row-level partitioning of a time series, not properties of RAN telemetry.

3) Per-bs continuous-feature distribution differences (Step 2 measurement): We measured pairwise per-bs Kullback–Leibler divergence on every continuous feature (`scripts/step2_mechanism_search.py`, output `artifacts/step2_mechanism_search.json`). The top-5 most-discriminating features by mean pairwise bs-KL are `dl_cqi` (KL = 0.475), `dl_mcs` (0.267), `num_ues` (0.092), `tx_brate_dl_Mbps` (0.081), and `sum_granted_prbs` (0.059). The top two — channel-quality indicator (CQI) and modulation-and-coding scheme (MCS) — are physical-layer features that depend on each gNB's specific RF environment and user-cell distance distribution, and their per-bs distributions differ by an order of magnitude more than the next-most-discriminating features. The remaining continuous features (BLER, RSSI, SINR, buffer occupancy) all have mean pairwise bs-KL below 0.02, confirming that the bs-discriminative signal is concentrated in channel-state and load features rather than in BLER itself. These per-bs feature differences are real, but the run-level controls (Section VII-A2) show they are *not* the causal mechanism: `run_random` spreads exactly these per-bs continuous features across clients (breaking bs-coherence) yet matches natural-by-BS to within paired CI_{95} $[-0.0003, +0.0001]$, so the channel-state distributional signal is not what the natural partition exploits. The operative axis is the temporal contiguity of each run's sliding windows (sequence integrity; Section VII-A2), which natural-by-BS preserves and every row-level partition (Dirichlet / `random_split`) destroys.

4) Applicability boundary: The inverted-α_{dir} finding is dataset-structural, not algorithm-mechanistic; the conditions under which it should replicate to other near-RT RIC forecasting workloads are detailed in App. A.1.

5) Hardware drift caveat for the random-split ablation: Section VII-A1 cells ran on 4× Tesla V100-SXM2-32GB while Phase 5 ran on RTX 4080. We bound the cross-hardware AUC drift by comparing V100 `random_split` AUC to 4080 Dirichlet $\alpha_{\text{dir}} = 5.00$ AUC (both partition strategies destroy bs grouping): the residual delta is at most 0.0072 (LSTM), 0.0043 (Mamba), 0.0021 (Spiking-SSM), all within seed σ. Hardware drift is therefore bounded above by ∼ 0.007 AUC, more than an order of magnitude smaller than the ∼ 0.18 mechanism signal (ratio ≈ 25×). Full bf16-tensor-core argument and protocol detail in App. A.2.

6) Per-BS Dirichlet ablation: within-BS row-scatter corrupts sequences too (supports the sequence-integrity mechanism): Section VII-A1's `random_split` ablation could not separate candidate (i) ("natural-by-BS preserves bs-conditioned signal") from (ii) ("Dirichlet's per-slice row redistribution destroys structurally-coherent client datasets") because `random_split` violates both. We test the **strong reading of (i)** — that bs grouping *alone* suffices, regardless of within-bs slice handling — via a per-BS Dirichlet ablation (`per_bs_dirichlet`, `sub_per_bs=2`, $\alpha_{\text{dir}} \in \{0.05, 0.5, 5, 10\}$, 5 seeds × 3 archs = 60 cells, 4× V100): each BS is split into two sub-clients via $\text{Dirichlet}([\alpha_{\text{dir}}])$ over `slice_id`; no client sees rows from multiple BSs. At $\alpha_{\text{dir}} = 0.05$ sub-clients are slice-specialists; at $\alpha_{\text{dir}} = 10$ each has near-uniform slice mixture. Headline numbers in Table VIII.

TABLE VIII
PER-BS DIRICHLET ABLATION (PHASE 6, SUB_PER_BS=2, 5 SEEDS × 3 ARCHS). WITHIN-PHASE-6 Δ AND CI_{95} PAIRED BY SEED VIA 10 000-SAMPLE BOOTSTRAP (PAIRED CIS PARTIALLY CONTROL THE ADDITIVE COMPONENT OF PER-TEST-TR VARIANCE SHARED ACROSS THE CONTRASTED ALGORITHMS; SEE SECTION VIII L15 FOR THE RESIDUAL ALGORITHM-×-TR-INTERACTION CAVEAT); NATURAL-BY-BS COLUMN IS UNPAIRED ACROSS n AND HARDWARE. **ABSOLUTE-AUC VALUES SHOWN ARE SEED-ONLY MEAN; CLUSTER-AWARE CI PER SECTION VIII L15 WIDENS THE ABSOLUTE-AUC BAND BY 4–28×, BUT THE PAIRED Δ WITHIN PHASE 6 IS UNAFFECTED.**

arch	$\alpha_{\text{dir}} = 0.05$	$\alpha_{\text{dir}} = 10$	Δ (pp)	95% CI (pp)	natural-by-BS
LSTM	0.906	0.831	7.5	[6.6, 8.5]	0.916
Mamba	0.909	0.836	7.2	[6.8, 7.7]	0.917
Spiking	0.816	0.684	13.2	[11.7, 15.1]	0.853

Phase 6 (this paper's per-BS Dirichlet ablation, parallel naming to **Phase 5**'s Stage-2 sweep) means over 5 seeds; Δ and CI_{95} are *paired by seed within Phase 6* via a 10 000-sample bootstrap ($n = 5$ caveat: bootstrap percentile CIs at this n are mildly optimistic, but the effect-size-to-bias ratio is $\sim 10\times$ across all three archs). All three within-Phase-6 deltas exclude zero, *and remain non-zero under Bonferroni-corrected* $CI_{98.33}$ *for the 3-arch family* (LSTM $[6.5, 8.6]$, Mamba $[6.7, 7.8]$, Spiking $[11.5, 15.3]$ pp), so multiplicity correction does not change the verdict. Since bs grouping is invariant across these cells (each shard from exactly 1 BS), AUC degradation can only originate from per-client structural changes. **These data confirm the sequence-integrity mechanism (Section VII-A2).** `per_bs_dirichlet` keeps bs grouping intact yet still scatters rows *within* each BS via the per-slice Dirichlet, corrupting the sliding windows; its α_{dir}-dependence is the same fragmentation-severity effect as Section VI-B — at concentrated $\alpha_{\text{dir}} = 0.05$ one sub-client retains most of each run (AUC near natural), at uniform $\alpha_{\text{dir}} = 10$ each run is finely scattered (AUC degraded). bs grouping is therefore neither necessary (`run_random` breaks it with no loss) nor the operative axis; sequence integrity is.

The residual gap between Phase 6 $\alpha_{\text{dir}} = 0.05$ and the Phase 5 natural-by-BS baseline (LSTM 1.0, Mamba 0.8, Spiking 3.7 pp) is *unpaired across* n *and hardware* (Phase 5 $n = 10$ on RTX 4080; Phase 6 $n = 5$ on V100; cf. Section VII-A5 hardware-drift bound ≤ 0.007 AUC). It is consistent with two design constraints (`sub_per_bs=2` halves per-client data; per-round sampling fraction drops $5/7 \to 5/14$) but cannot be cleanly decomposed; the within-Phase-6 effect (7–13 pp) dominates this residual.

7) Quantifying the tr-embedding-bug confound: A code-level audit identified that the test traffic-config index `tr` $\in \{25, 26, 27\}$ is encoded via `nn.Embedding(30, 8)`, but the train set only ever indexes rows $\{0..21\}$. Rows 22–29 therefore stay at PyTorch's default $\mathcal{N}(0, 1)$ initialisation and are bit-identical to a fresh seed-0 init at test time (verified across all 3 architectures by the `tr`-embedding byte-identity invariant in `tests/test_audit_invariants.py`). The concern this raises: this random-vector test-time confound might artificially inflate natural-by-BS dominance if the Dirichlet partition relies more on the `tr` feature.

We quantify the impact via inference-only re-evaluation on 54 cells (3 archs × FedAvg × {natural-by-BS, Dirichlet $\alpha_{\text{dir}} = 0.05$} × 9 seeds; `experiments/run_p1_tr_embedding_check.py --archs lstm mamba spiking_expand2`) — load each Phase 5 checkpoint, replace `tr`-embedding rows 22–29 with the mean of trained rows 0–21, re-compute test AUC. Per-architecture summary in Table IX.

TABLE IX
TR-EMBEDDING-BUG CONFOUND ACROSS 3 BACKBONES. "SHRINKAGE" IS $(\Delta\text{gap}_{\text{normal}} - \Delta\text{gap}_{\text{meanfix}})/\Delta\text{gap}_{\text{normal}}$. ALL 3 ARCHS SHOW $\leq 10.2\%$ BUG CONTRIBUTION; RESIDUAL GAPS REMAIN 0.04–0.15 AUC.

arch	gap_normal	gap_meanfix	shrinkage	residual
LSTM	+0.0540	+0.0491	9.2%	0.0491
Mamba	+0.0477	+0.0428	10.2%	0.0428
Spiking-SSM	+0.1580	+0.1545	2.3%	0.1545

The bug explains $\leq 10\%$ of natural-by-BS dominance across all 3 backbones; the remaining $\geq 90\%$ is structural, consistent with the Section VII-A1 `random_split` + Section VII-A6 per-BS Dirichlet ablations.

The Spiking-SSM result is particularly informative: it has the largest absolute natural-by-BS gap (0.16 AUC) but the smallest relative bug contribution (2.3%) — i.e., on the architecture where C1's effect is strongest, the bug confound is weakest. This rules out the alternative reading "natural-by-BS dominance is mostly a tr-embedding artefact". The reviewer Minor#4 confound is real but immaterial to C1's substantive claim. The bug is a fixable artefact of the embedding-table sizing convention; recommended fix discussed in `artifacts/audit/tr_embedding_audit.md`.

OOD design clarification. Because train `tr` $\in \{0..21\}$ and test `tr` $\in \{25..27\}$, the model *never observes test* `tr` *embeddings during training*, so at test time the `nn.Embedding(30, 8)` lookup for `tr` $\in \{25..27\}$ returns the original PyTorch $\mathcal{N}(0, 1)$ initialised vectors (verified bit-identical to fresh seed-0 init in `tests/test_audit_invariants.py`). This is a real OOD design weakness: from the model's perspective the test-time `tr` feature is effectively replaced by random noise. The 9.2/10.2/2.3% shrinkage estimates above already incorporate this random-input effect by construction (the meanfix replaces rows 22–29 with the mean of trained rows 0–21, which is the closest "informative" substitute to a random vector). The fact that the natural-by-BS gap remains 90–98% intact under meanfix means the C1 finding does *not* depend on the model exploiting a structured signal in the test-`tr` embedding — the structural lift comes from the other categorical (`slice_id`, `sched`) and the 17 continuous channel-state features.

Empirical confirmation via no-`tr` ablation. We

re-trained LSTM × FedAvg × natural-by-BS × 10 seeds × 100 rounds with the `tr` embedding removed entirely (`drop_categorical=["tr"]` on the encoder; spec `experiments/specs/r2_no_tr_ablation.yaml`). Paired-by-seed test AUC: no-`tr` 0.9165 ± 0.0004 vs Phase 5 with-`tr` 0.9159 ± 0.0005; mean paired Δ (no-`tr` − with-`tr`) = $+\mathbf{0.0006}$ AUC, paired-bootstrap CI_{95} $[+\mathbf{0.0003}, +\mathbf{0.0010}]$ ($n_{\mathrm{boot}} = 10\,000$, $n = 10$). The CI excludes zero in the *positive* direction — removing the buggy `tr` embedding doesn't just preserve performance, it slightly *improves* it (8/10 seeds no-`tr` ≥ Phase 5; 2/10 equal; 0/10 worse). This empirically confirms the meanfix-based $\leq 10\%$ bug / $\geq 90\%$ structural quantification: the buggy `tr` embedding contributed essentially zero signal at test time (consistent with the random-input semantics) and the natural-by-BS dominance survives intact when it is removed. **The empirical robustness statement is therefore: C1 mechanism finding for the LSTM × FedAvg × natural-by-BS configuration is robust to the embedding choice** (no-`tr` ≥ Phase 5 with-`tr` at the seed level); for Mamba (10.2% shrinkage) and Spiking (2.3% shrinkage), the meanfix proxy above remains the basis of the $\geq 90\%$ structural claim, and the same no-`tr` ablation on those two backbones is open future work that would tighten the cross-arch claim (Appendix D). A future-work replacement of `tr` with the underlying RBG-allocation numeric vector is no longer needed for C1's defensibility on LSTM; if added, it would be a slight implementation tightening (Appendix D). Source: `artifacts/r2_no_tr_ablation/aggregated.json`.

B. Algorithmic flatness implies FedAdam is the ceiling, not the start

Section VI-C establishes that on this dataset the FedAdam-vs-FedAvg gap (Dirichlet-averaged $\approx +0.006$ to $+0.016$ AUC depending on architecture) bounds the empirical maximum gap that the 5 server-side aggregation algorithms in our preregistered baseline extract under our (7-client, 71% participation, 100-round) regime. The Section VII-E FedSWA add-on test extends this empirical ceiling by $+0.000379$ AUC at LSTM × natural-by-BS — well within the $+0.016$ envelope, so the bound holds empirically even though FedSWA marginally exceeds FedAdam. The flatness is *not* a property of FL in general — large-N low-participation FL on image classification regularly shows ≥ 0.05 AUC gaps between FedAvg and FedSAM/FedSWA-class methods. The flatness is a property of *aggregation-noise-dominated* small-N high-participation regimes where the per-round update $(v_t - \theta_{t-1})$ reflects SGD noise across nearly-overlapping client averages rather than coherent drift toward sharp minima.

This explains the negative result on FedDyn-canonical: the canonical Acar 2021 SGD-friendly variant $h_i \leftarrow h_i + (w_l - w_g)$ diverges to NaN under our Adam local optimiser at the 100-round budget, due to a positive-feedback loop between Adam's adaptive scaling and the unbounded h-accumulation. Our Adam-friendly Option-II variant ($h_i \leftarrow h_i - \alpha_{\mathrm{dyn}} \cdot \nabla \mathcal{L}_i(w_t)$ with $\alpha_{\mathrm{dyn}} = 0.01$) reaches centralised parity (test AUC 0.9147 on LSTM IID at 100 rounds, matching the centralised reference). The lesson generalises: server-side aggregation algorithms that perform well in their original SGD regime can require non-trivial recalibration under Adam local optimisers, and the absence of such recalibration is a frequent silent failure mode in the FL benchmark literature.

C. The Mamba × SCAFFOLD × $\alpha_{\mathrm{dir}} = 0.10$ catastrophic interaction

The Section VI-D finding warrants an explicit mechanism account. SCAFFOLD's per-client control variate $c_{\mathrm{local},i}$ is a running estimate of the client's average gradient direction relative to the global average, and the gradient correction $+(c_{\mathrm{global}} - c_{\mathrm{local},i})$ rotates each local update toward an "unbiased" trajectory in expectation. Under high partition skew ($\alpha_{\mathrm{dir}} = 0.10$) the per-client gradient direction is itself highly variable across rounds (each client trains on a small, sparse-positive subset of the train rows), so $c_{\mathrm{local},i}$ is a noisy estimate. The selective-scan SSM in Mamba additionally has an input-dependent dynamics matrix ($\Delta t, B, C$ are all functions of the input — Gu and Dao [37]), making the per-client gradient direction *also* depend on the local partition's input distribution in a way the LSTM's input-independent recurrence does not. The combination — noisy $c_{\mathrm{local},i}$ × input-dependent backbone dynamics — produces a feedback loop where the SCAFFOLD correction overshoots in the direction of the prior-round's noise, and the next round's local training amplifies the overshoot rather than correcting it.

Mamba × FedAvg at the same α_{dir} does not exhibit this failure (std 0.025 at $\alpha_{\mathrm{dir}} = 0.10$ vs 0.083 under SCAFFOLD), nor does LSTM × SCAFFOLD (std 0.036 at $\alpha_{\mathrm{dir}} = 0.10$) where the input-independent recurrence breaks the feedback. The pattern is therefore specific to the *intersection* of input-dependent recurrence and noisy control-variate estimation; it does not generalise to all (state-space-model, control-variate) pairs but should be re-checked whenever SCAFFOLD or its variants are deployed alongside selective state-space architectures (S4, S5, S6, Mamba-2 — the input-dependent SSM family generally) under partition skew. The SCAFFOLD hyperparameter envelope (client-LR, control-variate decay) was not exhaustively swept; see Section VIII for the limitation.

D. Architecture leverage dominates algorithm leverage on commodity edge GPU

The Section VI-E Pareto observation has direct deployment implications. On RTX 4080 (sm_89, TSMC 4N (5 nm class) Lovelace process, 320 W TGP), the 10× energy band between LSTM ($\sim$ 3.5 kJ/cell) and Spiking-SSM ($\sim$ 41 kJ/cell) is dominated by the spiking arm's $T = 5$ simulation timesteps multiplied through the dense post-spike linear layers — the so-called spike-count amplification documented in our prior centralized findings. On a sparsity-aware accelerator (Loihi-2, IBM NorthPole) where post-spike multiplications by zero are skipped, the same architecture would sit closer to the LSTM band; the energy-Pareto disposition between LSTM and Spiking is therefore a property of the deployment hardware, not of the algorithm choice. We discuss the cross-hardware extrapolation in Section VIII.

TABLE X
FEDSWA EMPIRICAL COMPARISON VS PHASE 5 FEDADAM + FEDAVG ON LSTM × NATURAL-BY-BS, 5 PAIRED SEEDS. PAIRED-BOOTSTRAP CI_{95} ON FEDSWA − FEDADAM EXCLUDES ZERO IN THE POSITIVE DIRECTION (5/5 SAME-DIRECTION).

Algorithm	Test AUC mean	vs FedAvg paired Δ
FedAvg	0.915779	—
FedAdam	0.917622	+0.001843
FedSWA ($\alpha_{\mathrm{LA}} = 1.5$)	**0.918001**	**+0.002222**

The within-architecture algorithm spread (≤ 0.03 AUC at flat energy) is small enough that a deployed system should select algorithm based on operational considerations (state size — FedDyn and SCAFFOLD carry per-client h or c variates; communication round count; client straggler tolerance) rather than chase the $+0.01$ AUC gain that FedAdam offers over FedAvg. The Section VI-D catastrophic-interaction caveat is the principal exception: deployments combining selective-SSM backbones with control-variate algorithms should validate on representative high-skew partitions before committing to the combination.

E. FedSWA empirical comparison + revised mechanism framing

The 2024–2026 sharpness-aware FL family (FedSWA [48], FedSCAM [12], FedMoSWA [14]) is structurally adjacent to FedAdam in that all four algorithms modify the server-side aggregation step. Section II-F developed a mechanism-based argument that LookAhead-style overshoot is dominated by Adam-style adaptive variance damping in our aggregation-noise-dominated regime, predicting $|\Delta_{\mathrm{FedSWA-FedAvg}}| < |\Delta_{\mathrm{FedAdam-FedAvg}}|$. We ran the empirical comparison: 5 paired seeds × LSTM × FedSWA ($\alpha_{\mathrm{LA}} = 1.5$ per Liu et al. eq. 17) × natural-by-BS × 100 rounds; results in Table X.

Paired-bootstrap CI_{95} (FedSWA − FedAdam, $n_{\mathrm{boot}} = 10\,000$, $n = 5$ paired seeds): $\Delta = \mathbf{+0.000379}$ $[+0.000206, +0.000553]$. **5/5 seeds same direction; CI excludes zero in the positive direction.** ($n = 5$ caveat: bootstrap percentile CIs at this n are mildly optimistic in the sense of Section VII-A6; here the lower CI bound $+0.000206$ is the conservative tail. We treat the qualitative "5/5 same direction, CI_{95} excludes zero" reading as primary and the magnitude as illustrative; an $n = 10$ paired re-run is open future work.) The $\alpha_{\mathrm{LA}} = 1.0 \rightarrow$ FedAvg reduction of our FedSWA implementation is unit-tested up to float32 roundoff in `tests/test_fedswa_methodology.py`, so the $+0.000379$ paired Δ is not contaminated by an "FedSWA at $\alpha_{\mathrm{LA}} = 1.0 \neq$ FedAvg" implementation artefact.

Empirical revision of the Section II-F mechanism prediction. The directional inequality is REVERSED at natural-by-BS in this regime: FedSWA's gain over FedAvg ($+0.0022$) marginally exceeds FedAdam's ($+0.0018$). However, the magnitude of the FedSWA-vs-FedAdam advantage ($+0.0004$ AUC) is *(a)* an order of magnitude smaller than the FedAdam-vs-FedAvg gap of $+0.0018$ *at natural-by-BS*, *(b)* $\sim 150\times$ smaller than the inter-architecture gap of $+0.06$ AUC (Section VI-E), and *(c)* within the per-test-config noise floor $\sigma_{\mathrm{tr}} \approx 7.5\mathrm{e}{-3}$ documented in Section VIII L15. The Section II-F mechanism argument is directionally wrong but quantitatively correct *at natural-by-BS*: server-side aggregation modifications saturate at a tight band ~ 0.002 AUC over FedAvg here, and FedSWA's LookAhead-EMA achieves its reported gain through a different mechanism than FedAdam's adaptive variance damping but lands in the same neighbourhood. **Scope of "tight band ~ 0.002": this number applies at natural-by-BS specifically.** Per Section VI-C, the FedAdam-vs-FedAvg gap widens to $+0.006$–$+0.016$ AUC under Dirichlet $\alpha_{\mathrm{dir}} \in \{0.05..5.0\}$; whether the FedSWA-vs-FedAdam advantage similarly widens at high heterogeneity is untested in this work and is open future work (a follow-up FedSWA × {LSTM, Mamba, Spiking} × Dirichlet sweep would clarify this; we explicitly do *not* claim FedSWA universally tracks FedAdam to within 0.0004 AUC across all partitions). The Section I contribution 2 framing ("algorithmic leverage is small") is consistent with this $+0.0004$ AUC FedSWA-vs-FedAdam offset: server-side LookAhead-style modifications stay within the FedAdam-over-FedAvg envelope at the tested cell. This implementation omits the FedSWA paper's cyclical local LR schedule (eq. 3); whether that component would tighten or widen the FedSWA−FedAdam gap is open future work.

Per-cell + paired-bootstrap data: `artifacts/r34_fedswa_natural/aggregated_vs_fedadam.json`. FedMoSWA empirical evaluation remains deferred to follow-up work; FedSCAM and FedGMT are empirically evaluated in Section VII-F.

F. FedSCAM and FedGMT empirical comparison

We close the Section VII-E deferral for FedSCAM and FedGMT [13] with a paper-pinned-hyperparameter sweep on 4× Tesla V100-SXM2-32GB: LSTM × {FedSCAM, FedGMT} × {IID, Dirichlet $\alpha_{\mathrm{dir}} \in \{0.05, 0.10, 0.50, 1.0, 5.0\}$} × 5 seeds = 60 cells (132 min wall, 0 NaN). FedSCAM uses paper Algorithm 1 mid-of-tested values: $\rho_{\max} = 0.05$, $\alpha_\rho = 1.0$, $\gamma = 1.0$, $\beta_{\mathrm{align}} = 0.8$, $\kappa = 1.0$, $B_{\mathrm{pilot}} = 3$; FedGMT uses the `harrylee999/FL-SAM` reference impl example: $\alpha_{\mathrm{EMA}} = 0.95$, $\gamma_{\mathrm{KL}} = 1.0$, $\tau = 3.0$, $\beta = 10$. Paired-bootstrap CI_{95} on the 5-seed per-partition AUC delta vs Phase 5 baselines (matching the Section IV-E protocol):

FedSCAM finds a moderate-heterogeneity niche. The largest effect is at $\alpha_{\mathrm{dir}} = 0.10$: paired $\Delta = +0.0092$ AUC $[+0.0025, +0.0185]$, 5/5 same direction, CI excludes zero. Small positive effects at $\alpha_{\mathrm{dir}} \in \{1.00, 5.00\}$ ($+0.0012$–$+0.0018$, CIs excluding zero but ≤ 0.002 AUC). At IID, $\alpha_{\mathrm{dir}} = 0.05$, and $\alpha_{\mathrm{dir}} = 0.50$ the paired CI_{95} overlaps zero — FedSCAM ties FedAvg. The SAM perturbation pays off in the moderate-heterogeneity band where per-client gradient norms are large enough to drive non-trivial ρ_i modulation (paper Algorithm 1: $\rho_i = \rho_{\max}/(1 + \alpha_\rho h_i^{\mathrm{adj}})$) but the partition is not so extreme that the pilot direction s_i misaligns with the previous global direction u_{t-1}.

FedGMT is strictly dominated by FedAdam across all six partitions. Paired $\Delta \in [-0.0271, -0.0053]$ AUC, 0/5

TABLE XI
FEDSCAM VS FEDAVG AND FEDGMT VS FEDADAM, LSTM, 5 PAIRED SEEDS PER PARTITION, $n_{\text{boot}} = 10\,000$. "$+/n$" IS THE SAME-DIRECTION COUNT. † MARKS CELLS WHERE CI_{95} EXCLUDES ZERO.

Partition	FedSCAM − FedAvg	$+/n$	FedGMT − FedAdam	$+/n$
IID	$+0.0003$ $[-0.0000, +0.0006]$	4/5	-0.0053 $[-0.0065, -0.0042]^\dagger$	0/5
$\alpha_{\text{dir}} = 0.05$	$+0.0023$ $[-0.0019, +0.0066]$	3/5	-0.0260 $[-0.0374, -0.0147]^\dagger$	0/5
$\alpha_{\text{dir}} = 0.10$	$+0.0092$ $[+0.0025, +0.0185]^\dagger$	5/5	-0.0271 $[-0.0399, -0.0146]^\dagger$	0/5
$\alpha_{\text{dir}} = 0.50$	$+0.0028$ $[-0.0027, +0.0084]$	4/5	-0.0145 $[-0.0190, -0.0104]^\dagger$	0/5
$\alpha_{\text{dir}} = 1.00$	$+0.0018$ $[+0.0008, +0.0029]^\dagger$	5/5	-0.0180 $[-0.0242, -0.0119]^\dagger$	0/5
$\alpha_{\text{dir}} = 5.00$	$+0.0012$ $[+0.0003, +0.0022]^\dagger$	4/5	-0.0109 $[-0.0155, -0.0075]^\dagger$	0/5

same-direction (all five seeds negative) on every partition, CI_{95} excludes zero negatively on every partition. The largest deficits are at $\alpha_{\text{dir}} \in \{0.05, 0.10\}$ (-0.026 to -0.027 AUC), the same heterogeneity band where FedSCAM extracts its $+0.0092$. The Section II-F mechanism prediction — that variance-blind sharpness extrapolation cannot beat FedAdam's adaptive variance damping in our aggregation-noise-dominated regime — is empirically confirmed for FedGMT but *refuted* for FedSCAM at $\alpha_{\text{dir}} = 0.10$ (where FedSCAM's per-client SAM-radius modulation contributes signal that FedAdam's server-side moment estimation does not capture).

KL-term ablation (4060 Ti, paired, IID, 5 seeds). To isolate the contribution of FedGMT's KL-distillation term against the Global Model Trajectory (Eq. (9) of [13]), we re-ran the IID arm with $\gamma_{\text{KL}} = 0$ (KL term short-circuits in the client-side closure; the loss reduces to BCE $+\frac{1}{\beta}\langle w, h_{\text{dual}}\rangle$). On the same 5 seeds at sample-ratio 0.1 for both arms (the cross-arm pairing was the design fix; absolute AUC is therefore lower than the V100 sweep): mean $\Delta_{\text{KL}} = \text{AUC}_{\gamma=1} - \text{AUC}_{\gamma=0} = -0.0025 \pm 0.0014$, 0/5 positive (5/5 same-direction *negative*). The KL term marginally *hurts* on this dataset in the IID regime — consistent with the FedGMT-vs-FedAdam result that the KL anchor against the EMA trajectory does not produce useful additional signal beyond what FedAdam's (m, v) second-moment estimation already captures. Source: `artifacts/v7_fedgmt_kl_off/` + `artifacts/v7_fedgmt_kl_on/` + `/tmp/sam_paired_ci.json`.

Mechanism revisited. The Section II-F prediction that sharpness-aware FL methods land within FedAdam's $+0.006$–$+0.016$ envelope (Section VI-C) survives in three out of four senses: (a) FedSWA in Section VII-E stays within the envelope at natural-by-BS, (b) FedGMT in this section stays within the envelope on the *losing* side ($\Delta \le -0.0053$ AUC at IID, widening to -0.027 at $\alpha_{\text{dir}} = 0.10$ — still inside the ± 0.016 band magnitude-wise at IID but exceeds it on the negative side at heterogeneous partitions), (c) the KL-ablation confirms FedGMT's underperformance is mechanism-tied (KL term hurts in 5/5 IID seeds). The prediction *fails* in one sense: FedSCAM's $+0.0092$ AUC gain at $\alpha_{\text{dir}} = 0.10$ exceeds the FedAdam-vs-FedAvg gap at the same partition ($+0.005$ AUC per Section VI-C), indicating per-client SAM-radius modulation is a genuinely orthogonal signal to server-side adaptive variance damping for moderate heterogeneity. The empirical result is therefore a calibrated 1-of-3 niche win for the sharpness-aware family, in the specific cell (LSTM, $\alpha_{\text{dir}} = 0.10$, 5 seeds).

Honest caveats. (i) Both algorithms run under Adam ($\eta = 5\text{e}{-4}$) in our v7 pipeline; the paper versions of FedSCAM and FedGMT both use SGD with momentum 0.9. Canonical FedDyn diverged under Adam at 100 rounds in our prior centralized work; the NaN guard added to `_local_loop.py` (`NonFiniteLossError`, raised when `loss.item()` is non-finite) was a defensive measure for FedGMT's structurally-similar dual term but never triggered (0/60 cells, 0/10 ablation cells NaN). (ii) FedSCAM's K-means clustering and `Proj_d` random projection (paper Algorithm 1 lines marked "optional") are not implemented in our in-tree port; our ~ 44K-parameter LSTM does not benefit from $d \in \{256, 512\}$ projection. (iii) Our FedGMT impl follows the reference `harrylee999/FL-SAM` repository for the EMA-update timing (server round end), which differs by a one-round phase shift from paper Algorithm 1 line 7 (per-client round start). The two share the same recurrence $e_t = \alpha e_{t-1} + (1-\alpha) w_t$. (iv) Wilcoxon signed-rank p-values at $n = 5$ paired seeds floor at 0.0625 (the most extreme outcome of 5/5 same-direction); the CI_{95} exclusion of zero is therefore primary evidence here, and a follow-up $n = 10$ paired re-run on the cells flagged with † in Table XI is open future work. (v) The 4060 Ti KL-ablation used `sample_ratio=0.1` (host-RAM constraint at the 4060 Ti's 30 GB vs V100's 754 GB); the cross-arm pairing on the same data slice ensures the Δ_{KL} measurement is internally valid, but absolute AUCs (~ 0.66 at IID) are not comparable to the V100 sweep's full-data values (~ 0.91).

Per-cell summary JSONs: `artifacts/v7_sam_family/v7_*/summary.json`. Aggregated paper-grade table: `docs/RESULTS_V7_SAM_FAMILY.md`. Paired-CI95 JSON: `/tmp/sam_paired_ci.json`. Implementations: `src/fl_oran/federated/algorithms/{fedscam.py,fedgmt.py}`.

G. Recommended practice: a partitioning checklist for federated time-series benchmarks

The sequence-integrity artifact is not specific to our pipeline: it arises whenever a federated benchmark *synthesises* client heterogeneity by row-level partitioning (Dirichlet over samples, or uniform random assignment — the de-facto standard since NIID-Bench) of a *pooled* time series and then constructs sliding windows per client. The row-level split scatters each entity's consecutive timesteps across clients, so per-client windows span temporal gaps; any sequence model degrades, and

a sequence-preserving partition then looks spuriously superior. We distil a short checklist for FL-time-series benchmarking:

1) **Partition by entity/sequence, not by row.** Assign whole entities (or whole `(run, slice)` series) to clients; never shuffle rows before windowing.
2) **Verify window contiguity.** Report the fraction of per-client sliding windows whose timesteps are contiguous; it should be 1.0 (cf. the 0.001–0.84 values for row-level Dirichlet, Section VI-B).
3) **Include a sequence-preserving control.** When comparing a natural/entity partition against a synthetic non-IID partition, add a *run-level* version of the synthetic partition (same skew, intact sequences). If the gap vanishes (as here), the effect was sequence corruption, not heterogeneity.
4) **Separate the axes.** Vary heterogeneity and entity-coherence independently of sequence integrity (e.g., `run_random` and `run_dirichlet`) before attributing an effect to "client heterogeneity."

Our `run_random` / `run_dirichlet` partition modes (released with the benchmark) implement items 3–4. Applying this checklist to our own data turns a striking "inverted heterogeneity" into a corrected, mundane result — FL on O-RAN slice-SLA is robust to client heterogeneity once sequences are kept intact — and we conjecture the same audit would revise heterogeneity effects in other federated time-series benchmarks that partition by row.

VIII. Limitations and Threats to Validity

We enumerate the limitations and threats to the validity of the Section I contributions. Each item is paired with the contribution(s) it most threatens.

- **L1 (threats contributions 1, 4): Single-hardware energy reporting.** All energy numbers are NVML measurements on a single RTX 4080 (sm_89). Energy on consumer-tier mobile GPUs (RTX 30-series at lower TGP), data-centre accelerators (A100, H100), and sparsity-aware accelerators (Loihi-2) will differ in absolute magnitude and possibly in the architectural ranking. The headline $10\times$ LSTM-vs-Spiking energy ratio is a property of the dense GPU + $T = 5$ spiking-simulation regime; on sparsity-aware hardware the ratio would shrink (prior centralized estimate ≈ 0.49). A cross-GPU robustness sweep (T4, A100) is open future work.
- **L2 (threats contributions 2, 3): main 5-algorithm baseline excludes 2024–2026 sharpness-aware FL; FedSWA add-on in Section VII-E; FedSCAM + FedGMT 60-cell extension in Section VII-F.** FedSWA [48] is empirically evaluated in Section VII-E (5-seed paired bootstrap on LSTM $\times$ natural-by-BS); the result marginally reverses the Section II-F directional prediction but FedSWA's gap over FedAvg ($+0.0022$) lands well within the $+0.016$ envelope Section VI-C establishes for the preregistered baseline. FedSCAM [12] and FedGMT [13] are empirically evaluated in Section VII-F (60 cells V100, 5-seed paired bootstrap on LSTM $\times$ 6 partitions); FedSCAM finds a $+0.0092$ AUC niche at $\alpha_{\text{dir}} = 0.10$ (CI_{95} excludes zero) but ties FedAvg at the other five partitions, while FedGMT is strictly dominated by FedAdam at all six partitions ($\Delta \in [-0.027, -0.005]$ AUC, all CI_{95} exclude zero negatively). The Section II-F mechanism prediction survives for FedGMT but is partially refuted for FedSCAM at the moderate-heterogeneity cell — per-client SAM-radius modulation extracts signal that FedAdam's server-side (m, v) does not capture. FedMoSWA [14] empirical evaluation on Colosseum/ColO-RAN remains deferred to follow-up work; the mechanism-based argument in Sections II-F + VI-C + VII-B predicts it would perform within the FedAdam $+0.016$ envelope in our regime. Regarding FedBN [11], often suggested as the feature-shift FL baseline: our 3 backbones (LSTM, Mamba, Spiking-SSM) intentionally omit BatchNorm. FedBN's defining server-side rule — skip BatchNorm parameters during aggregation — therefore reduces bit-exactly to FedAvg's complete aggregation on these architectures (proof in `artifacts/audit/fedbn_reduces_to_fedavg.md`). **We additionally ran a 30-cell empirical confirmation** (3 archs $\times$ FedBN $\times$ natural-by-BS $\times$ 10 seeds, 100 rounds, RTX 4080) and verified the reduction on the trained outputs. Per-architecture paired-bootstrap CI_{95} ($n_{\text{boot}} = 10\,000$, matching the Section IV-E protocol) on the (FedBN − Phase-5-FedAvg) per-seed test-AUC delta: LSTM $\Delta = 0$ $[0, 0]$ (bit-exact 10/10 seeds); Mamba $\Delta = +1.0\text{e}{-6}$ $[-4\text{e}{-6}, +8\text{e}{-6}]$; Spiking $\Delta = +4.9\text{e}{-4}$ $[-4.4\text{e}{-3}, +4.3\text{e}{-3}]$. **All 3 architectures CI_{95} include zero** — empirically equivalent to FedAvg. The Spiking CI width ($\sim 9\text{e}{-3}$) is wider than the inter-algorithm gaps reported in Section VI-C (FedAdam-vs-FedAvg $\approx$ 0.006–0.016 AUC), but this BF16-numerical-noise floor affects all algorithm-pair comparisons equally and is captured in Section VI-C's paired-bootstrap CIs by the same mechanism. A 3-cell diagnostic further isolates the Spiking-arch $\sim 3.5\text{e}{-3}$ cross-time delta as environmental drift over the 6 weeks between Phase 5 and the FedBN sweep (likely CUDA driver / PyTorch / matmul-order changes; BF16 numerics are not bit-reproducible across such updates), not algorithmic difference. FedAvg is already in our 5-algorithm baseline. The reported FedBN benefit on cellular feature-skew tasks (arXiv:2508.08479 [8] reports $+0.01$–0.03 AUC over FedAvg) is BN-specific and structurally not transferable to no-BN backbones. **Beyond the literal FedBN-reduces-to-FedAvg proof, we additionally tested the FedBN-style personalisation principle directly:** take each Phase 5 LSTM/Mamba/Spiking $\times$ FedAvg $\times$ natural-by-BS checkpoint, per BS fine-tune $K = 200$ Adam steps on local train data with the same hyperparameters as Phase 5 client step, then evaluate per-BS personalised vs global AUC on the OOD test split. Across 15 (arch $\times$ seed) cells $\times$ 7 BS each $= 105$ per-BS observations: mean $\Delta_{\text{personalised}-\text{global}} = -0.0025$ AUC. The cross-observation std (0.0036) is dominated by the per-arch mean spread (Spiking pulls down the

cross-arch mean); the **within-arch** stds across seeds are LSTM $4e{-}4$, Mamba $2e{-}4$, Spiking $3e{-}3$ — i.e. individual fine-tunes are an order of magnitude tighter than the cross-observation std suggests. Per-arch means: LSTM -0.0007, Mamba -0.0014, Spiking -0.0053. **0/15 cells show positive personalisation Δ at the cell level** — this simple per-BS fine-tuning operationalisation of feature-shift personalisation (a BN-independent reading of the FedBN-spirit) does *not* improve test-AUC on this dataset across the 3 backbones tested; we do not generalise this to all FedBN-style or feature-shift personalisation methods. Source: `artifacts/r2_post_hoc_per_bs_finetune/aggregated.json` + `experiments/run_r2_post_hoc_per_bs_finetune.py`. This more directly addresses the "most directly relevant feature-shift FL baseline is missing" concern and supports the Section VI-B mechanism reading that the global FedAvg model is already optimal for the natural-by-BS partition. MOON [59] is structurally deferred (preregistered) because its `forecaster_encode_fn` does not generalise cleanly to selective-SSM or spiking-SSM activations.

- **L3 (threats contribution 1): Round count is short by image-classification standards.** We use 100 communication rounds $\times$ 50 max-steps per round $\times$ 5 clients sampled per round $=$ **25 000 total gradient-steps per cell** (the figure that matches the audit-corrected centralised budget from our prior centralized work). Per individual client, each client is sampled with probability $5/7$ per round and executes 50 steps when sampled, so the *expected* per-client gradient-step count is $100\times(5/7)\times50 \approx 3\,571$ steps per client per cell. Both numbers are short relative to the 1 000-round runs typical of FedSWA / FedSCAM evaluations. The predicted-flatness argument holds at this round count; whether it relaxes at longer round counts is open. A 1 000-round LSTM $\times$ FedAvg $\times$ IID extrapolation experiment is cheap ($\sim$ 3 hr GPU on RTX 4080) and is a candidate future ablation.
- **L4 (threats contribution 1): 7-client natural-deployment scale.** All experiments use 7 clients — the natural base-station count of the public ColO-RAN release, not a researcher choice — at $5/7 = 71\%$ participation per round. Statements about FL behaviour at the cross-device 1 000-client / 10% participation regime [60] do not generalise from this study. The Section VII-B argument explicitly acknowledges this: contribution 2's flatness is a property of high-participation FL at small-N (here, the dataset-imposed $N = 7$), not of FL in general. Larger-scale ColO-RAN releases or alternative O-RAN testbeds (e.g., OpenAirInterface 5G, srsRAN) could let future work probe the cross-device limit, subject to non-trivial engineering effort to align telemetry format with our preprocessing pipeline (OAI 5G's logging differs substantially; srsRAN lacks integrated FL training infrastructure).
- **L5 (threats contribution 5): Reproducibility infrastructure not yet released.** The full 4-axis table maps deployment configurations to AUC / F1 / energy across 90 groups; the contribution 5 reproducibility claim depends on the Section V release of the spec-driven YAML pipeline + paired-bootstrap statistics + Croissant metadata + Zenodo DOI under AGPL-3.0. At submission time the artefact is mature in `src/fl_oran/`, `experiments/specs/`, and `tests/` but the Section V reproducibility section is staged for the camera-ready release; we will provide a Dockerfile + `requirements.lock` + demo notebook in the supplementary archive.
- **L6 (threats contribution 4): Pareto Spiking arm assumes hardware sparsity unavailable on RTX 4080.** The Section VI-E architecture-leverage claim treats Spiking as a distinct vertical band at $\sim$ 41 kJ/cell. On the deployed sparsity-aware target the Spiking arm's energy would compress (per Section VII-D and prior centralized analysis), and the Pareto envelope between LSTM and Spiking would tighten substantively. Operators targeting commodity GPU should read the Section VI-E ranking as-is; operators targeting Loihi-2 / NorthPole should read the ranking with a 2–3$\times$ downward correction on Spiking energy.
- **L7 (was "mechanism open"; now RESOLVED, Section VII-A2): the inverted-α_{dir} is a sequence-integrity artifact.** The `random_split` ablation (Section VII-A1) could not separate bs-coherence from sequence integrity because it breaks both at once. The run-level controls (Section VII-A2) resolve it: `run_random` (intact runs, bs-shuffled) ties natural-by-BS (paired CI_{95} $[-0.0003, +0.0001]$) and run-level Dirichlet holds AUC flat across α_{dir}, so the inversion is per-client sliding-window corruption introduced by row-level partitioning, not a structural property. Residual scope: the run-level controls are LSTM-primary (Mamba / Spiking-SSM confirmed in Table VII) on a single dataset; cross-dataset replication is open future work.
- **L17 (scope of the sequence-integrity pitfall): benchmark-construction-specific.** The artifact arises specifically when heterogeneity is *synthesised* by row-level partitioning of a pooled time series followed by per-client windowing (the standard FL-benchmark recipe). Real deployments with naturally-partitioned entity clients (each gNB holding its own intact stream) do not incur it; our recommendation to partition by entity is the deployment-correct choice and trivial in situ. The generality claim is therefore about *benchmarking methodology*; per-paper susceptibility of specific prior benchmarks depends on their exact partition-then-window order, which we have not audited individually.
- **L8 (threats contribution 2): SCAFFOLD interaction not exhaustively swept.** The Section VI-D catastrophic-interaction finding uses canonical SCAFFOLD hyperparameters. A SCAFFOLD client-LR / control-variate-EMA grid sweep on Mamba $\times$ $\alpha_{\text{dir}} = 0.10$ would establish whether the failure is recoverable in some hyperparameter region; we report the interaction with canonical hyperparameters to establish the existence of the failure mode rather than its precise envelope.

TABLE XII
LOTO CLUSTER-BOOTSTRAP VARIANCE DECOMPOSITION (LSTM/MAMBA/SPIKING × FEDAVG × NATURAL-BY-BS, 10 SEEDS × 3 TEST `TR` CONFIGS). EXTERNAL UNCERTAINTY σ_{tr} EXCEEDS INTERNAL σ_{init} BY 2.3–17.8×; CLUSTER CI_{95} WIDTHS EXCEED STANDARD SEED-PAIRED CI_{95} WIDTHS BY 3.7–28×. WITH ONLY 3 TEST `TR` CONFIGS, THE CLUSTER CI IS SOMEWHAT CONSERVATIVE; MORE CONFIGS WOULD TIGHTEN IT WHILE PRESERVING THE DIRECTIONAL RESULT.

arch	σ_{init}	σ_{tr}	ratio $\sigma_{\text{tr}}/\sigma_{\text{init}}$	CI95 width ratio
LSTM	4.2e−4	7.5e−3	17.8×	28×
Mamba	5.6e−4	7.3e−3	13.0×	20×
Spiking-SSM	5.1e−3	1.2e−2	2.3×	3.7×

- **L9 (threats Sections III, VI, VII-A): `pos_weight_split=train` is one of three valid choices.** The BCE loss reweighting is globally pooled across all clients (Section VII-A), so the *direction* of all findings is invariant to this choice; full alternative-computation discussion in App. C.1.
- **L10 (threats contribution 5 reproducibility): bf16 mixed-precision training versus fp32.** All Phase 5 cells use bf16; we do not report an fp32 verification cell. Full numerical-precision argument in App. C.2.
- **L11 (threats contribution 1, Section VII-A): Per-client compute budget split as 100 rounds × 50 max-steps versus alternative splits.** We treat round count as a fixed reproducibility anchor; whether the Section VI-C FedAdam-saturates-headroom finding survives at alternative splits is open. Full discussion in App. C.3.
- **L12 (threats contribution 1): SLA-threshold sensitivity not exhaustively swept.** The 10% BLER threshold is the canonical ColO-RAN SLA gate [1]; a $\{5\%, 15\%, 20\%\}$ sweep is the highest-leverage open ablation after Section VII-A1. Full discussion in App. C.4.
- **L13 (threats contribution 5 reproducibility): Bootstrap CI uses percentile interval, not BCa.** At our $n = 10$ paired seeds with approximately symmetric per-seed delta distributions, the BCa bias-correction term is bounded by $O(1/\sqrt{n}) \approx 0.32$ standard errors — smaller than seed σ on every reported finding except Section VI-D's Mamba×SCAFFOLD. Full justification in App. C.5.
- **L14 (threats contribution 2): FedAdam hyperparameter sensitivity untested.** $\beta_1 = 0.9$, $\beta_2 = 0.99$, $\eta_{\text{server}} = 0.01$ follow Reddi et al. 2021 [56] preregistered values; $(\beta_1, \beta_2, \eta_{\text{server}})$ sensitivity sweep deferred. Full discussion in App. C.6.
- **L15 (cluster-level external validity).** Standard 10-seed paired-bootstrap CI_{95} captures init-RNG noise but not external uncertainty across test traffic configurations. We ran a leave-one-traffic-config-out (LOTO) cluster bootstrap on the LSTM/Mamba/Spiking × FedAvg × natural-by-BS Phase 5 cells (10 seeds × 3 test `tr` configs each, inference-only, `experiments/run_p2_loto_cluster_bootstrap.py`): see Table XII.
  For absolute test-AUC claims (per-cell numbers in Section VI tables), the standard CI_{95} is 4–28× too tight — though the qualitative conclusions survive because the inverted-α_{dir} gap (~ 0.17 AUC, Section VI-B) and the architecture-band separation (~ 0.06–0.07 AUC, Section VI-E) dwarf even the cluster-CI_{95} widths (≤ 0.023 AUC). **For paired-delta claims** (Section VI-C FedAdam-vs-FedAvg, Section VI-D Mamba×SCAFFOLD, the 270 paired-bootstrap distributions), the per-test-config noise *partially* cancels per-pair: both algorithms in a contrast are evaluated on the same fixed `tr` split, so the additive component of σ_{tr} that is shared between the two algorithms drops out of the paired delta. Residual non-cancellation comes from any algorithm × `tr` interaction (an algorithm whose ranking flips across `tr` configs); we have not measured this third-order effect, so the paired-bootstrap CI_{95} should be interpreted as an internal-noise CI under the assumption that algorithm × `tr` interaction is small relative to σ_{init}. The headline Section VI-D Mamba×SCAFFOLD finding (paired $\Delta = -0.0915$, CI_{95} $[-0.1288, -0.0544]$, 10/10 seeds same direction) is robust under the cluster framework. Per-arch LOTO data in `artifacts/p2_loto/results*.json`.
- **L16 (FL is not formal privacy).** FL $\neq$ formal privacy. Sharing only weight updates / gradients (rather than raw telemetry) reduces data-locality but does not constitute a formal privacy guarantee: model-update reconstruction attacks ("Deep Leakage from Gradients" [61]) demonstrate that shared gradients can reconstruct training inputs in some regimes. Differential privacy (per-client DP-SGD), secure aggregation (cryptographic gradient masking), and gradient-leakage hardening are all compatible with the FedAvg/FedAdam pipeline used here but are out of scope for this benchmark — the paper's contribution is the architecture × algorithm × heterogeneity sweep, not a privacy mechanism evaluation. Operators deploying federated SLA prediction should pair the chosen algorithm with a privacy mechanism whose threat-model matches the operator's policy.

IX. CONCLUSION

We have presented the first comprehensive 4-axis federated-learning benchmark on the publicly-available Colosseum/ColO-RAN testbed for slice-level SLA-violation prediction, sweeping {LSTM, Mamba, Spiking-SSM} × {FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn} × {natural-by-BS, Dirichlet $\alpha_{\text{dir}} \in [0.05, 5.0]$} × 10 seeds = 900 cells with NVML idle-baseline-subtracted training-energy measurement on commodity-tier RTX 4080, augmented by a 15-cell V100 ablation that tests the leading mechanism hypothesis for the headline inverted-α_{dir} finding.

The principal finding is methodological: **the apparent "inverted heterogeneity" — natural-by-BS seemingly beating every Dirichlet α_{dir} across all architectures and algorithms — is a sequence-integrity artifact of row-level non-IID partitioning, not a structural property of RAN telemetry.** Controlled run-level partitioning experiments (Section VII-A2) show that breaking bs-coherence while keeping each run's sliding windows intact (`run_random`) costs $\sim$0 AUC, and

a run-level Dirichlet at matched α_{dir} holds AUC flat at the natural-by-BS level, while the row-level deficit tracks window-fragmentation severity (the per-client contiguous-window fraction rank-correlates perfectly with AUC). Because row-level Dirichlet partitioning is the de-facto FL-benchmark default and is routinely transplanted to time-series tasks, this pitfall plausibly affects other federated time-series benchmarks; Section VII-G distils a partitioning checklist. The deployment recommendation — partition by natural gNB — is unchanged and trivial (it preserves each cell's time series); what does not survive is the scientific reading that RAN telemetry carries special cell-conditional structure defeating standard FL heterogeneity. The three secondary findings (FedAdam saturating algorithmic headroom; Mamba × SCAFFOLD catastrophic interaction; architecture leverage dominating algorithm leverage on energy) are detailed in Section I contributions 2–4 and Sections VI-C–VI-E; we do not restate the numbers here.

The community-level implication is the one operationalisable insight beyond the per-cell numbers: standard FL practice — *partition clients by inducing artificial uniformity, then deploy sharpness-aware aggregation to suppress the heterogeneity-induced noise* — is the wrong protocol for O-RAN edge-FL when natural client structure (one gNB, one client) is already heterogeneous in the prediction-relevant axis. The recommendation, in our setup, reverses: **preserve the natural client partition, and require synthetic re-partitioning to be justified by a specific operational constraint** (e.g., privacy-aware aggregation) rather than adopted as a default benchmarking convention. For O-RAN edge-FL practitioners, the operational recommendation is concrete: deploy on the natural base-station partition rather than reshuffle clients to enforce IID-like uniformity; select architecture for the energy-AUC trade-off (LSTM at the low-energy frontier on commodity GPU; Spiking-SSM remains a research direction pending sparsity-aware accelerator availability — its $10\times$ higher energy on dense GPU is hardware-specific, not a fundamental architectural property, and a deployment recommendation must wait until target neuromorphic / sparsity-aware hardware is empirically characterised, see Section VII-D + Section VIII L1, L6); choose algorithm based on operational constraints (state size, communication overhead, straggler tolerance) rather than chase ≤ 0.02 AUC gains, with the explicit caveat that selective-SSM × control-variate combinations need partition-aware validation before deployment.

Upon paper acceptance, we will release the full benchmark artefact — spec-driven YAML pipeline, pre-flight algorithm validation, paired-bootstrap statistical inference, NVML training-energy instrumentation, Croissant dataset metadata, and a Zenodo-archived release tag — as a reference baseline for further FL-on-O-RAN research.

A. Future work

Five open directions, each detailed in App. D:

- **Mechanism isolation re-run** — `sub_per_bs=2` with `clients-per-round=10` to disentangle slice-mixing from per-round participation in the Section VII-A6 residual gap, in line with the broader non-IID-impact methodology of Jimenez-Gutierrez et al. [62] and the proximity-partition framing of ProFed [63].
- **BLER-threshold sensitivity** — perturb the 10% gate to $\{5, 15, 20\}\%$ to confirm operational robustness [64].
- **Scaling beyond** $N = 7$ — test whether inverted-α_{dir} holds at larger client counts in the 6G slicing / edge-FL setups studied by Chiarani et al. [65] and Sezgin et al. [66].
- **Variance-aware sharpness** — FedSCAM [12] and FedSynSAM [67] as candidates beyond the FedAdam ceiling (Section VII-B).
- **Privacy + neuromorphic** — over-the-air DP [68], sparse-RNN edge acceleration [69], and spiking-NN low-power edge sensing [70].

REFERENCES

[1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "ColO-RAN: Developing machine learning-based xApps for Open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, 2022.

[2] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, and A. Talwalkar, "LEAF: A benchmark for federated settings," *arXiv:1812.01097*, 2018.

[3] F. Lai, Y. Dai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "FedScale: Benchmarking model and system performance of federated learning at scale," in *ICML*, 2022.

[4] C. Song, F. Granqvist, and K. Talwar, "FLAIR: Federated learning annotated image repository," 2022.

[5] F. Granqvist, C. Song, Á. Cahill, R. van Dalen, M. Pelikan, Y. S. Chan, X. Feng, N. Krishnaswami, V. Jina, and M. Chitnis, "pfl-research: Simulation framework for accelerating research in private federated learning," in *NeurIPS Datasets and Benchmarks*, 2024, arXiv:2404.06430.

[6] S. Shen, D. Zhao, L. Feng, Z. Yue, J. Li, T. Li, G. Shen, and Y. Zeng, "STEP: A unified spiking transformer evaluation platform for fair and reproducible benchmarking," in *NeurIPS*, 2025, arXiv:2505.11151.

[7] L. Pu, Q. Cui, X. Li, B. Zhao, W. Ni, M. Ai, and X. Tao, "Federated learning-based heterogeneous load prediction and slicing for 5G systems and beyond," in *IEEE GLOBECOM*, 2022, pp. 166–172.

[8] Y. Dutta, S. Chatterjee, S. Chakraborty, and B. Palit, "FedAvg/FedProx/FedBN benchmark on five throughput-prediction datasets," 2025, arXiv:2508.08479.

[9] P. Mangi, S. Bibi, A. Nawaz, and S. Bibi, "When Clients Drift: Federated SLA-risk forecasting across unseen 6G RAN regimes," *Spectrum of Engineering Sciences*, 2026, no arXiv preprint located 2026-05-08.

[10] M. Polese, L. Bonati, S. D'Oro, P. Johari, D. Villa, S. Velumani, R. Gangula, M. Tsampazi, C. P. Robinson, G. Gemmi, A. Lacava, S. Maxenti, H. Cheng, and T. Melodia, "Colosseum: The open RAN digital twin," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 5452–5466, Aug. 2024, arXiv:2404.17317.

[11] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *ICLR*, 2021, arXiv:2102.07623.

[12] S. Rahil, Z. A. Ahmad, and T. Asif, "FedSCAM: Federated sharpness-aware minimization with clustered aggregation and modulation," 2026, arXiv:2601.00853 (Jan 2026); per-client SAM radius modulation + alignment-aware aggregation. [Online]. Available: https://arxiv.org/abs/2601.00853

[13] Y. Li, T. Liu, Y. Cui, M. Hu, and X. Li, "One arrow, two hawks: Sharpness-aware minimization for federated learning via global model trajectory," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, openReview ID 80mK2Mqaph; reference implementation https://github.com/harrylee999/FL-SAM.

[14] J. Liu, Y. Liu, F. Shang, H. Liu, J. Liu, and W. Feng, "Improving generalization in federated learning with highly heterogeneous data via momentum-based stochastic controlled weight averaging (FedMoSWA)," in *ICML*, 2025, arXiv:2507.20016 (companion algorithm to Liu2025_FedSWA; same paper, separate cite key for the momentum variant).

[15] H.-C. Tsai, "FedRMamba: Federated residual Mamba for multivariate time-series forecasting," in *Proceedings of the ACM Web Conference (WWW)*. ACM, 2026, concurrent method-proposal: personalised client architecture combining a global frequency-aware Mamba module with a local patch-wise Mamba module for federated multivariate time-series forecasting.

[16] M. Beck, K. Pöppel, M. Spanring, A. Auer, O. Prudnikova, M. Kopp, G. Klambauer, J. Brandstetter, and S. Hochreiter, "xLSTM: Extended long short-term memory," in *NeurIPS*, 2024, arXiv:2405.04517. Introduces exponential input gate + normalizer state + stabilizer state. Two variants: sLSTM (scalar memory, used here) and mLSTM (matrix memory, deferred — not needed at our seq_len=5).

[17] A. Lahoti, K. Y. Li, B. Chen, C. Wang, A. Bick, J. Z. Kolter, T. Dao, and A. Gu, "Mamba-3: Improved sequence modeling using state space principles," arXiv:2603.15569, 2026, three innovations over Mamba-2: (1) exponential-trapezoidal discretization with data-dependent λ_t, (2) complex-valued SSM via RoPE-style 2×2 rotation (data-dependent θ_t), (3) MIMO state updates. We implement (1) + (2); (3) is an LLM-scale hardware optimization not needed at our 40K-param scale. Authors include Tri Dao and Albert Gu, the original Mamba and Mamba-2 authors.

[18] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," 2020, arXiv:2007.14390.

[19] O. Shchur, A. F. Ansari, C. Turkmen, L. Stella, N. Erickson, P. Guerron, M. Bohlke-Schneider, and Y. Wang, "fev-bench: A realistic benchmark for time series forecasting," 2025, arXiv:2509.26468.

[20] H. Chergui, L. Blanco, and C. Verikoukis, "Statistical federated learning for beyond-5G SLA-constrained RAN slicing," *IEEE Transactions on Wireless Communications*, vol. 21, no. 3, pp. 2066–2076, 2022.

[21] ——, "CDF-aware federated learning for low SLA violations in beyond 5G network slicing," in *IEEE ICC*, 2021.

[22] J. Groen, Z. Yang, D. Muruganandham, M. Belgiovine, L. Ying, and K. Chowdhury, "From classification to optimization: Slicing and resource management with TRACTOR," 2023, arXiv:2312.07896.

[23] R. Barker, A. Ebrahimi Dorcheh, T. Seyfi, and F. Afghah, "REAL: Reinforcement learning-enabled xApps for experimental closed-loop optimization in O-RAN with OSC RIC and srsRAN," in *IEEE ICC Workshops*, 2025, arXiv:2502.00715.

[24] R. J. Hayek, K. Comer, J. Chung, C. R. Murthy, R. Kettimuthu, and I. Kadota, "Beyond assumptions: Measuring federated learning over real 5G networks," 2025, arXiv:2504.04678.

[25] J. Chen, Y. Yang, S. Deng, D. Teng, and L. Pan, "SpikMamba: When SNN meets Mamba in event-based human action recognition," in *6th ACM International Conference on Multimedia in Asia (MMAsia)*, 2024, arXiv:2410.16746.

[26] J. Qin and F. Liu, "Mamba-Spike: Enhancing the Mamba architecture with a spiking front-end for efficient temporal data processing," in *Computer Graphics International (CGI)*, 2024, arXiv:2408.11823.

[27] P. Wu, B. Chai, M. Zheng, W. Li, Z. Hu, J. Chen, Z. Zhang, H. Li, and X. Sun, "Efficient spiking point Mamba for point cloud analysis," in *ICCV*, 2025, arXiv:2504.14371.

[28] S. Shen, C. Wang, R. Huang, Y. Zhong, Q. Guo, Z. Lu, J. Zhang, and L. Leng, "SpikingSSMs: Sparse-parallel spiking state-space models," in *AAAI*, vol. 39, no. 19, 2025, pp. 20 380–20 388, arXiv:2408.14909.

[29] Y. Huang, J. Tang, C. Wang, Z. Wang, J. Zhang, Z. Lu, B. Cheng, and L. Leng, "SpikingMamba: Towards energy-efficient large language models via knowledge distillation from Mamba," *Transactions on Machine Learning Research (TMLR)*, 2026, arXiv:2510.04595.

[30] Y. Pan, Y. Feng, J. Zhuang, S. Ding, H. Xu, Z. Liu, B. Sun, Y. Chou, X. Qiu, A. Deng, A. Hu, S. Wang, P. Zhou, M. Yao, J. Wu, J. Yang, G. Sun, B. Xu, and G. Li, "SpikingBrain: Spiking brain-inspired large models," 2025, arXiv:2509.05276.

[31] Y. Venkatesha, Y. Kim, L. Tassiulas, and P. Panda, "Federated learning with spiking neural networks," *IEEE Transactions on Signal Processing*, 2021, arXiv:2106.06579.

[32] D. Yu, X. Du, L. Jiang, H. Zhang, and S. Deng, "FedLEC: Federated learning with spiking neural networks under label-skew non-iid," in *IJCAI*, 2025, arXiv:2412.17305.

[33] M. Abbasihafshejani, A. Maiti, and M. Jadliwala, "Spiking neural networks in vertical federated learning: Performance trade-offs," 2024, arXiv:2407.17672.

[34] M. V. Nguyen, L. Zhao, B. Deng, and S. Wu, "The robustness of spiking neural networks in federated learning with compression against non-omniscient Byzantine attacks," 2025, arXiv:2501.03306.

[35] D. Aksu, J. Martinez del Rincon, and I. Alouani, "Privacy in federated learning with spiking neural networks," 2025, arXiv:2511.21181.

[36] L. Pereira, M. Perkusich, D. Valadares, and K. Gorgônio, "Firing-rate sensitivity to differential privacy in federated spiking neural networks," in *ICASSP*, 2026, arXiv:2602.12009 (Feb 2026).

[37] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *COLM*, 2024, arXiv:2312.00752.

[38] G. Shen, D. Zhao, T. Li, J. Li, and Y. Zeng, "Is conventional SNN really efficient? a perspective from network quantization," in *CVPR*, 2024, arXiv:2311.10802; venue: CVPR 2024.

[39] A. Asperti, D. Evangelista, and M. Marzolla, "Dissecting FLOPs along input dimensions for GreenAI cost estimations," in *LOD*, 2021, arXiv:2107.11949.

[40] J. Yik, K. Van den Berghe, D. den Blanken, Y. Bouhadjar, M. Fabre, P. Hueber, W. Ke, M. A. Khoei, D. Kleyko, N. Pacik-Nelson, A. Pierro, P. Stratmann, P.-S. V. Sun, G. Tang, S. Wang, B. Zhou, V. J. Reddi, C. Frenkel *et al.*, "NeuroBench: A framework for benchmarking neuromorphic computing algorithms and systems," *Nature Communications*, 2025, arXiv:2304.04640; published Feb. 11, 2025.

[41] J.-W. Chung, J. J. Ma, R. Wu, J. Liu, O. J. Kweon, Y. Xia, Z. Wu, and M. Chowdhury, "The ML.ENERGY benchmark: Toward automated inference energy measurement and optimization," in *NeurIPS Datasets and Benchmarks (Spotlight)*, 2025, arXiv:2505.06371.

[42] J.-W. Chung, R. Wu, J. J. Ma, and M. Chowdhury, "Where do the joules go? diagnosing inference energy consumption," 2026, arXiv:2601.22076.

[43] P. Spyra, "Beyond backpropagation: Exploring innovative algorithms for energy-efficient deep neural network training," 2025, arXiv:2509.19063.

[44] Q. Li, Y. Diao, Q. Chen, and B. He, "NIID-Bench: Federated learning on non-IID data silos: An experimental study," in *ICDE*, 2022, arXiv:2102.02079.

[45] C. J. Mclaughlin and L. Su, "Personalized federated learning via feature distribution adaptation," in *NeurIPS*, 2024, arXiv:2411.00329.

[46] K. Borazjani, P. Abdisarabshali, N. Khosravan, and S. Hosseinalipour, "Redefining non-IID data in federated learning for computer vision tasks: Migrating from labels to embeddings for task-specific data distributions," *IEEE Transactions on Artificial Intelligence*, 2026, arXiv:2503.14553.

[47] D. Caldarola, B. Caputo, and M. Ciccone, "Improving generalization in federated learning by seeking flat minima (FedSAM)," in *ECCV*, 2022.

[48] J. Liu, Y. Liu, F. Shang, H. Liu, J. Liu, and W. Feng, "FedSWA: Improving generalization in federated learning with highly heterogeneous data via stochastic controlled weight averaging," in *ICML*, 2025, arXiv:2507.20016.

[49] K. Ali, Z. Bettouche, A. Kassler, and A. Fischer, "Enhancing spatiotemporal networks with xLSTM: A scalar LSTM approach for cellular traffic forecasting," arXiv:2507.19513, 2025, closest prior art to our xLSTM panel: applies the sLSTM block to *centralized* cellular traffic forecasting (Jul 2025). We differ in being federated and in targeting O-RAN slice-SLA prediction rather than traffic volume — this paper delimits the gap our xLSTM extension fills.

[50] T. Raviv and N. Shlezinger, "WiMamba: Linear-scale wireless foundation model," arXiv:2603.26367, 2026, mamba-based wireless foundation model (Mar 2026): evidence that selective state-space models are an active wireless direction, motivating our Mamba / Mamba-3 panel.

[51] H. Mohammadi, S. B. Hashemi Natanzi, R. Nassiri, J. Hassanpour, B. Tang, and V. Marojevic, "SliceFed: Federated constrained multi-agent DRL for dynamic spectrum slicing in 6G," arXiv:2603.11390, 2026, contemporary (Mar 2026) federated O-RAN slicing work; uses multi-agent DRL for spectrum-slicing *control* — a control task, distinct from our supervised slice-SLA *prediction* benchmark. Cited to position scope.

[52] WiNES Lab, Northeastern University, "colosseum-oran-coloran-dataset (public ColO-RAN release)," https://github.com/wineslab/colosseum-oran-coloran-dataset, 2024, repository hosting the unified per-second per-(bs, slice, sched, tr) telemetry table (7 base stations, 3 slices, 3 scheduling policies, 28 traffic configurations, Colosseum Rome scenario). All experiments in this paper use the public release without modification.

[53] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," 2019, arXiv:1901.09948; canonical surrogate-gradient reference underlying snntorch's `ATan` surrogate. The "Wei et al. 2018" attribution in source markdown corresponds to no verifiable paper; this Neftci 2019 survey is the authoritative reference for the surrogate- gradient method snntorch implements. Cite key retained to avoid main.tex churn.

[54] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[55] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks (FedProx)," in *MLSys*, 2020.
[56] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization (FedAdam)," in *ICLR*, 2021.
[57] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *ICML*, 2020.
[58] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization (FedDyn)," in *ICLR*, 2021.
[59] Q. Li, B. He, and D. Song, "Model-contrastive federated learning (MOON)," in *CVPR*, 2021.
[60] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," in *NeurIPS Workshop on Federated Learning*, 2019, arXiv:1909.06335.
[61] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, 2019, arXiv:1906.08935; demonstrates that shared gradients in collaborative training can reconstruct training inputs – motivates the §8 L16 caveat that FL is not formal privacy.
[62] D. M. Jimenez-Gutierrez, M. Hassanzadeh, A. Anagnostopoulos, I. Chatzigiannakis, and A. Vitaletti, "A thorough assessment of the non-IID data impact in federated learning," 2025, arXiv:2503.17070.
[63] D. Domini, G. Aguzzi, and M. Viroli, "ProFed: a benchmark for proximity-based non-IID federated learning," 2025, arXiv:2503.20618.
[64] E. Minus, R. Y. Coley, S. M. Shortreed, and B. D. Williamson, "Rare-event metric stability under threshold perturbation," 2025, arXiv:2504.16185.
[65] M. Chiarani, S. Roy, C. Verikoukis, and F. Granelli, "XAI-driven client selection for FL in scalable 6G network slicing," 2025, arXiv:2503.12435.
[66] Y. E. Sezgin, M. Özdem, and T. Bilen, "Federated edge learning for predictive maintenance in 6G small cell networks," 2025, arXiv:2509.11421.
[67] Y. Gu, R. Jin, Z. Zhang, and H. Dai, "Gradient compression may hurt generalization: A remedy by synthetic data guided sharpness aware minimization," 2026, arXiv:2602.11584; introduces the FedSynSAM method.
[68] H. Liang, H. Wen, K. Wu, and H. Xing, "Differential privacy as a perk: Federated learning over multiple-access fading channels with a multi-antenna base station," 2026, arXiv:2510.23463.
[69] A. Pierro, S. Abreu, J. Timcheck, P. Stratmann, A. Wild, and S. B. Shrestha, "Accelerating linear recurrent neural networks for the edge with unstructured sparsity," in *ICML*, 2025, arXiv:2502.01330.
[70] G. O. Y. L.-F. Lundstrom-Imanov and T. Yilmaz, "EdgeSpike: Spiking neural networks for low-power autonomous sensing in edge IoT architectures," 2026, arXiv:2604.27004.